



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده مهندسی کامپیوتر

پروژه کارشناسی
مهندسی کامپیوتر

طراحی و پیاده سازی سامانه ی واسط برای
یکپارچه سازی سرویس های هوش مصنوعی

نگارش

مصطفی رحمتی

استاد راهنما

دکتر حسین زینلی

بهار ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

صفحه فرم ارزیابی و تصویب پایان نامه – فرم تأیید اعضای کمیته

دفاع

در این صفحه فرم دفاع یا تأیید و تصویب پایان نامه موسوم به فرم کمیته دفاع – موجود در پرونده آموزشی – را قرار دهید.

نکات مهم:

- نگارش پایان نامه/پروژه باید به **زبان فارسی** و بر اساس آخرین نسخه دستورالعمل و راهنمای تدوین پایان نامه های دانشگاه صنعتی امیرکبیر باشد.
- رنگ جلد پروژه کارشناسی باید مشکی باشد.
- چاپ و صحافی پایان نامه/پروژه به صورت **پشت و رو (دورو)** بلامانع است و انجام آن توصیه می شود.

به نام خدا



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

تاریخ: بهار ۱۴۰۵

تعهدنامه اصالت اثر

اینجانب **مصطفی رحمتی** متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است. در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

مصطفی رحمتی

امضا

فهرست مطالب

صفحه	عنوان
۱	فهرست نمادها و اختصارات
۲	۱ مقدمه
۲	۱-۱ کلیات
۳	۲-۱ بیان مسئله
۴	۳-۱ انگیزه و ضرورت
۴	۴-۱ اهداف پروژه
۵	۵-۱ پرسش‌ها و معیارهای ارزیابی
۶	۶-۱ دستاوردهای پروژه
۷	۷-۱ تغییرات نسبت به پروپوزال
۷	۸-۱ ساختار پایان‌نامه
۸	۲ مروری بر پیشینه
۸	۱-۲ مقدمه
۸	۲-۲ مفاهیم پایه
۸	۱-۲-۲ سامانه‌ی واسط API
۹	۲-۲-۲ معماری خدمات‌گرا و ریزسرویس
۹	۳-۲-۲ مدل‌های زبانی بزرگ و سرویس‌های هوش مصنوعی
۱۰	۴-۲-۲ رابط‌های سازگار با OpenAI
۱۰	۳-۲ سیستم‌های مشابه

۱۰	 OpenAI	۱-۳-۲
۱۱	 Gemini	۲-۳-۲
۱۱	 Groq	۳-۳-۲
۱۱	 OpenRouter	۴-۳-۲
۱۲	 Anyscale Endpoints	۵-۳-۲
۱۲	 شکاف پژوهشی و معیارهای مقایسه	۴-۲
۱۳	 مقایسه و جمع‌بندی	۵-۲
۱۵	طراحی سیستم	۳
۱۵	 مقدمه	۱-۳
۱۵	 نیازمندی‌های طراحی	۲-۳
۱۷	 معماری کلی سیستم	۳-۳
۱۸	 جریان درخواست	۱-۳-۳
۱۸	 مدل داده	۴-۳
۱۸	 جداول احراز هویت و دسترسی	۱-۴-۳
۱۹	 جداول ارائه‌دهندگان و مدل‌ها	۲-۴-۳
۱۹	 جداول صورت‌حساب	۳-۴-۳
۱۹	 جداول گزارش مصرف	۴-۴-۳
۲۰	 طراحی API	۵-۳
۲۰	 نقاط پایانی احراز هویت و مدیریتی	۱-۵-۳
۲۰	 نقاط پایانی هوش مصنوعی	۲-۵-۳
۲۱	 الگوی خط لوله	۶-۳
۲۲	 استراتژی کش	۷-۳
۲۲	 طراحی سیستم صورت‌حساب	۸-۳
۲۴	پیاده‌سازی	۴
۲۴	 مقدمه	۱-۴
۲۴	 فناوری‌های استفاده‌شده	۲-۴

۲۴		چارچوب Laravel	۱-۲-۴
۲۵		پایگاه داده‌ی SQL Server	۲-۲-۴
۲۵		Redis	۳-۲-۴
۲۵		Vue.js	۴-۲-۴
۲۵		ساختار پوشه‌بندی پروژه	۳-۴
۲۶		پیاده‌سازی خط لوله‌ی سامانه‌ی واسط	۴-۴
۲۷		پیاده‌سازی آداپتورهای ارائه‌دهندگان	۵-۴
۲۷		OpenAI	۱-۵-۴
۲۷		Gemini	۲-۵-۴
۲۷		Groq و OpenRouter	۳-۵-۴
۲۸		پیاده‌سازی احراز هویت OTP	۶-۴
۲۸		پیاده‌سازی مدیریت کلیدهای API	۷-۴
۲۹		پیاده‌سازی سیستم صورت‌حساب	۸-۴
۲۹		پیاده‌سازی گزارش‌دهی و ارسال گزارش خارجی	۹-۴
۳۰		داشبورد Playground	۱۰-۴

۵ ارزیابی

۳۱		مقدمه	۱-۵
۳۱		روش ارزیابی	۲-۵
۳۲		محیط اجرا و داده‌های پایه	۱-۲-۵
۳۲		ارزیابی فنی	۳-۵
۳۲		آزمون‌های واحد	۱-۳-۵
۳۳		آزمون‌های ویژگی	۲-۳-۵
۳۳		نتایج آزمون	۳-۳-۵
۳۴		سناریوهای کلیدی تأییدشده	۴-۳-۵
۳۵		ارزیابی عملکرد	۴-۵
۳۵		تأخیر پاسخ	۱-۴-۵
۳۶		دقت صورت‌حساب‌گذاری	۲-۴-۵

۳۶	۵-۵	مقایسه با اهداف
۳۸	۶	نتیجه‌گیری
۳۸	۱-۶	جمع‌بندی
۳۸	۲-۶	دستاوردها
۳۹	۳-۶	محدودیت‌ها
۳۹	۴-۶	کارهای آتی
۴۳	آ	مستندات API
۴۳	۱-آ	قراردادهای مشترک
۴۴	۲-آ	نقاط پایانی احراز هویت
۴۴	۱-۲-آ	POST /api/v1/auth/otp/start
۴۵	۲-۲-آ	POST /api/v1/auth/otp/verify
۴۶	۳-آ	نقاط پایانی کاربر
۴۶	۱-۳-آ	GET /api/v1/me
۴۷	۲-۳-آ	PATCH /api/v1/me
۴۷	۴-آ	نقاط پایانی کلیدهای API
۴۷	۱-۴-آ	GET /api/v1/api-clients
۴۷	۲-۴-آ	POST /api/v1/api-clients
۴۸	۳-۴-آ	GET /api/v1/api-clients/{apiClient}/keys
۴۸	۴-۴-آ	POST /api/v1/api-clients/{apiClient}/keys
۵۰	۵-۴-آ	POST /api/v1/api-keys/{apiKey}/revoke
۵۰	۶-۴-آ	POST /api/v1/api-keys/{apiKey}/rotate
۵۰	۵-آ	نقاط پایانی هوش مصنوعی
۵۰	۱-۵-آ	POST /api/v1/ai/responses
۵۱	۲-۵-آ	POST /api/v1/ai/chat/completions
۵۳	۳-۵-آ	POST /api/v1/ai/embeddings
۵۴	۴-۵-آ	POST /api/v1/ai/images/generations

۵۵	POST /api/v1/ai/audio/transcriptions	۵-۵-آ
۵۶	POST /api/v1/ai/audio/speech	۶-۵-آ
۵۷	نقاط پایانی مالی	۶-آ
۵۷	GET /api/v1/wallet	۱-۶-آ
۵۷	POST /api/v1/wallet/topup	۲-۶-آ
۵۸	GET /api/v1/wallet/transactions	۳-۶-آ
۵۸	GET /api/v1/plans	۴-۶-آ
۵۹	POST /api/v1/plans	۵-۶-آ
۵۹	POST /api/v1/subscriptions	۶-۶-آ
۶۰	GET /api/v1/subscriptions/current	۷-۶-آ
۶۰	POST /api/v1/subscriptions/cancel	۸-۶-آ
۶۱	GET /api/v1/invoices	۹-۶-آ
۶۱	GET /api/v1/invoices/{invoice}	۱۰-۶-آ
۶۲	GET /api/v1/invoices/{invoice}/pdf	۱۱-۶-آ
۶۳	نقاط پایانی گزارش‌ها	۷-آ
۶۳	GET /api/v1/reports/usage	۱-۷-آ
۶۳	GET /api/v1/reports/wallet-ledger	۲-۷-آ
۶۴	GET /api/v1/reports/invoices	۳-۷-آ
۶۴	نقاط پایانی مدیریت ارائه‌دهندگان	۸-آ
۶۴	GET /api/v1/providers	۱-۸-آ
۶۴	POST /api/v1/providers	۲-۸-آ
۶۵	GET /api/v1/providers/{provider}/models	۳-۸-آ
۶۶	POST /api/v1/providers/{provider}/models	۴-۸-آ
۶۷	نقاط پایانی ممیزی	۹-آ
۶۷	GET /api/v1/audit-logs	۱-۹-آ

فهرست جداول

صفحه	جدول
۱۳	۱-۲ مقایسه‌ی سامانه‌های بررسی‌شده با مسئله‌ی این پایان‌نامه
۱۶	۱-۳ نیازمندی‌های اصلی مؤثر بر طراحی سامانه
۱۷	۲-۳ اجزای اصلی معماری سامانه
۳۲	۱-۵ مشخصات اجرای ارزیابی
۳۴	۲-۵ خلاصه‌ی اجرای مجموعه‌آزمون
۳۵	۳-۵ نمونه‌ای از سناریوهای کلیدی تأییدشده در آزمون‌ها
۳۷	۴-۵ مقایسه‌ی اهداف اولیه با وضعیت نهایی پیاده‌سازی

فهرست نمادها و اختصارات

معادل فارسی	معادل انگلیسی	اختصار
هوش مصنوعی	Artificial Intelligence	AI
رابط برنامه‌نویسی کاربردی	Application Programming Interface	API
مدل زبانی بزرگ	Large Language Model	LLM
رمز یک‌بار مصرف	One-Time Password	OTP
پروتکل انتقال ابرمتن	HyperText Transfer Protocol	HTTP
انتقال حالت بازنمایی	Representational State Transfer	REST
نمادگذاری شیء جاوااسکریپت	JavaScript Object Notation	JSON
رمزگذاری جفت‌بایت	Byte Pair Encoding	BPE
مدت اعتبار کش	Time To Live	TTL
کنترل دسترسی مبتنی بر نقش	Role-Based Access Control	RBAC
زبان پرس‌وجوی ساختاریافته	Structured Query Language	SQL
پیش‌پردازنده‌ی هایپرمتن	Hypertext Preprocessor	PHP
یکی‌کننده‌ی مکان منابع	Uniform Resource Locator	URL
پروتکل اینترنت	Internet Protocol	IP
تبدیل متن به گفتار	Text-to-Speech	TTS
تشخیص خودکار گفتار	Automatic Speech Recognition	ASR

فصل اول

مقدمه

۱-۱ کلیات

در دهه‌ی اخیر، هوش مصنوعی^۱ تحولی بنیادین در حوزه‌های مختلف فناوری اطلاعات ایجاد کرده است. ظهور مدل‌های زبانی بزرگ^۲ مانند GPT-4o، Gemini و Claude، امکانات بی‌سابقه‌ای را برای توسعه‌دهندگان نرم‌افزار و سازمان‌ها فراهم کرده است. این مدل‌ها از طریق رابط‌های برنامه‌نویسی کاربردی^۳ در دسترس عموم قرار گرفته‌اند و کاربردهای گسترده‌ای در حوزه‌هایی از جمله دستیاران هوشمند، تولید محتوا، تحلیل داده، تبدیل گفتار به متن و تولید تصویر یافته‌اند [۱۳].

شرکت‌های بزرگ فناوری از جمله Google، OpenAI، Gemini (با محصول Anthropic)، Claude (با محصول Groq) و سرویس‌های واسط مانند OpenRouter هر کدام رابط‌های برنامه‌نویسی منحصر به فرد خود را ارائه می‌دهند. گرچه بسیاری از این رابط‌ها قابلیت‌های مشابهی دارند، اما در ساختار درخواست، احراز هویت، فرمت پاسخ و سیستم صورت‌حساب‌گذاری تفاوت‌های قابل توجهی دارند [۸، ۱۴].

این تنوع باعث شده که سازمان‌هایی که قصد دارند از چند سرویس هوش مصنوعی به طور هم‌زمان بهره ببرند یا از یک ارائه‌دهنده به ارائه‌دهنده‌ی دیگری مهاجرت کنند، با چالش‌های فنی و مدیریتی قابل توجهی روبه‌رو شوند. در این شرایط، وجود یک سامانه‌ی واسط^۴ که بتواند ارتباط با سرویس‌های مختلف را به صورت استاندارد مدیریت کند، نیاز ملموسی است.

این پروژه با هدف طراحی و پیاده‌سازی چنین سامانه‌ای تعریف شده است. سامانه‌ی پیاده‌سازی شده به عنوان یک لایه‌ی میانجی بین سرویس‌گیرندگان^۵ و ارائه‌دهندگان مختلف هوش مصنوعی عمل می‌کند

^۱ Artificial Intelligence (AI)

^۲ Large Language Models (LLM)

^۳ Application Programming Interface (API)

^۴ Gateway

^۵ Clients

و یک رابط برنامه‌نویسی استاندارد و یکپارچه را در اختیار توسعه‌دهندگان قرار می‌دهد. علاوه بر این، قابلیت‌هایی مانند احراز هویت، مدیریت کلیدهای رابط برنامه‌نویسی، کیف پول، خرید اشتراک^۶، صدور فاکتور و گزارش‌های مصرف نیز در این سامانه پیش‌بینی شده است.

۱-۲ بیان مسئله

در اکوسیستم کنونی هوش مصنوعی، هر ارائه‌دهنده‌ی سرویس رابط برنامه‌نویسی مخصوص خود را تعریف می‌کند. برای مثال، OpenAI یک ساختار مشخص برای ارسال پیام و دریافت پاسخ دارد، Gemini از API بومی Google با فرمت متفاوتی استفاده می‌کند، و Groq اگرچه سازگار با فرمت OpenAI است، اما API Key و نقاط پایانی^۷ جداگانه‌ای دارد. این تفاوت‌ها در چند سطح مشکل‌ساز هستند:

در سطح فنی، توسعه‌دهنده‌ای که می‌خواهد از چند سرویس هوش مصنوعی استفاده کند ناچار است برای هر ارائه‌دهنده کد و منطق جداگانه‌ای بنویسد؛ همین موضوع حجم کد را افزایش می‌دهد و نگهداری سامانه را دشوارتر می‌کند. در نتیجه، مهاجرت از یک ارائه‌دهنده به دیگری نیز پرهزینه می‌شود، زیرا تغییر تنها به نشانی سرویس یا کلید دسترسی محدود نیست و معمولاً به بازنگری در ساختار درخواست، پاسخ و مدیریت خطا نیاز دارد.

در سطح مدیریتی و مالی نیز پراکندگی بیشتری دیده می‌شود. کلیدهای API، محدودیت نرخ^۸، گزارش مصرف و صورت‌حساب هر سرویس معمولاً به‌صورت مجزا مدیریت می‌شوند و همین امر تصویر یکپارچه‌ای از مصرف کل در اختیار سازمان قرار نمی‌دهد. افزون بر آن، بسیاری از ارائه‌دهندگان ابزار کافی برای مدیریت چندکاربره، کیف پول داخلی، صدور فاکتور یا خرید اشتراک ندارند. در نبود یک لایه‌ی مرکزی، کنترل دسترسی، تعیین سقف مصرف، ممیزی عملیات حساس و گزارش‌گیری دقیق باید برای هر سرویس به‌طور مستقل پیاده‌سازی شود.

این مشکلات با گسترش استفاده از هوش مصنوعی در سازمان‌ها حادتر می‌شود. راهکار پیشنهادی این پروژه، طراحی و پیاده‌سازی یک سامانه‌ی واسط است که تمامی این چالش‌ها را از طریق یک لایه‌ی مرکزی و یکپارچه حل کند.

Subscription^۶

Endpoints^۷

Rate Limit^۸

۱-۳ انگیزه و ضرورت

انگیزه‌ی اصلی این پروژه از نیاز واقعی سازمان‌ها و توسعه‌دهندگانی نشأت می‌گیرد که می‌خواهند از قابلیت‌های هوش مصنوعی بهره ببرند اما نمی‌خواهند درگیر پیچیدگی‌های یکپارچه‌سازی چندین سرویس مجزا شوند. ضرورت چنین سامانه‌ای را می‌توان از چند منظر بررسی کرد:

از منظر فنی، با داشتن یک رابط برنامه‌نویسی استاندارد و سازگار با OpenAI - که پرکاربردترین فرمت در این حوزه است - توسعه‌دهندگان می‌توانند بدون تغییر در کد سمت مشتری، از ارائه‌دهندگان مختلف استفاده کنند. تنها با تغییر فیلدهای provider و model در درخواست، سامانه درخواست را به ارائه‌دهنده‌ی مناسب هدایت می‌کند.

از منظر مالی، یک سیستم متمرکز مدیریت کیف پول و صورت حساب، امکان صدور فاکتور یکپارچه، ردیابی دقیق مصرف و کنترل هزینه‌ها را فراهم می‌کند. این موضوع برای سازمان‌هایی که هزینه‌های هوش مصنوعی بخش قابل توجهی از بودجه‌ی عملیاتی آن‌ها را تشکیل می‌دهد، از اهمیت ویژه‌ای برخوردار است.

از منظر امنیتی، متمرکزسازی احراز هویت و مدیریت کلیدهای API در یک سامانه‌ی واحد، امکان اعمال سیاست‌های امنیتی منسجم - مانند محدودیت نرخ درخواست، لیست سفید آدرس IP و ممیزی^۹ عملیات حساس - را به صورت مرکزی فراهم می‌آورد.

از منظر مقیاس‌پذیری، الگوی خط لوله^{۱۰} به‌کاررفته در این سامانه، امکان افزودن مراحل جدید به پردازش درخواست - مثلاً مدار قطعه^{۱۱} یا راهبری هوشمند^{۱۲} - را بدون تغییر در ساختار اصلی فراهم می‌کند.

۱-۴ اهداف پروژه

هدف اصلی این پروژه، طراحی و پیاده‌سازی یک سامانه‌ی واسط جامع برای یکپارچه‌سازی سرویس‌های هوش مصنوعی است. این هدف کلی به اهداف عملیاتی زیر تجزیه می‌شود. نخست، سامانه باید مجموعه‌ای

^۹ Audit

^{۱۰} Pipeline Pattern

^{۱۱} Circuit Breaker

^{۱۲} Smart Routing

از نقاط پایانی سازگار با استاندارد OpenAI را برای تکمیل گفتگو^{۱۳}، جاسازی متن^{۱۴}، تولید تصویر^{۱۵}، تبدیل گفتار به متن^{۱۶} و تبدیل متن به گفتار^{۱۷} در اختیار مشتری قرار دهد تا اتصال سرویس گیرندگان با حداقل تغییر ممکن انجام شود. در کنار این قرارداد بیرونی، پشتیبانی از چند ارائه دهنده از طریق آداپتور^{۱۸}های مجزا برای OpenAI، Gemini، Groq و OpenRouter نیز لازم است تا افزودن ارائه دهندگان جدید به صورت اتصال پذیر^{۱۹} ممکن بماند.

از سوی دیگر، پروژه فقط به لایه فنی محدود نیست. پیاده سازی جریان احراز هویت مبتنی بر رمز یک بار مصرف برای ورود به داشبورد مدیریتی، مدیریت کلیدهای API برای درخواست های برنامه ای، و یک سیستم مالی شامل کیف پول، ثبت دقیق مصرف توکن و منابع غیر توکنی، صدور فاکتور و مدیریت اشتراک های دوره ای از اهداف اصلی آن به شمار می روند. در نهایت، سامانه باید گزارش های تفصیلی مصرف، دفترکل تراکنش های کیف پول و فهرست فاکتورها را تولید کند، گزارش های ساخت یافته را به سرویس های بیرونی بفرستد و با تکیه بر آزمون های واحد و ویژگی و مستندات OpenAPI به اندازه ی کافی ارزیابی پذیر و قابل نگهداشت باشد.

۵-۱ پرسش ها و معیارهای ارزیابی

برای آن که مسیر استدلال این پایان نامه از تعریف مسئله تا ارزیابی نهایی شفاف باشد، سه پرسش اصلی زیر به عنوان محور ارزیابی در نظر گرفته شده اند:

۱. **پرسش اول:** آیا می توان یک رابط بیرونی سازگار با OpenAI ارائه کرد که ناهمگنی چند ارائه دهنده ی متفاوت - به ویژه ارائه دهنده های سازگار و ناسازگار با این قرارداد - را در لایه ی درونی پنهان کند؟

۲. **پرسش دوم:** آیا می توان در کنار یکپارچه سازی فنی، قابلیت های مدیریتی و مالی مورد نیاز یک سازمان - مانند احراز هویت، مدیریت کلید، کیف پول، اشتراک، فاکتور و گزارش مصرف - را نیز در همان درگاه تجمیع کرد؟

Chat Completions^{۱۳}

Embeddings^{۱۴}

Image Generation^{۱۵}

Audio Transcription^{۱۶}

Text-to-Speech^{۱۷}

Adapter^{۱۸}

Pluggable^{۱۹}

۳. پرسش سوم: آیا پیاده‌سازی انجام‌شده از نظر درستی جریان‌های بحرانی، قابلیت توسعه و انسجام معماری، شواهد کافی برای یک نمونه‌ی کارکردی و قابل اتکا ارائه می‌دهد؟

برای پاسخ به این پرسش‌ها، چهار معیار ارزیابی در این پایان‌نامه به کار گرفته شده است. معیار نخست، کامل بودن قرارداد بیرونی است؛ یعنی این که آیا نقاط پایانی اصلی هوش مصنوعی و مسیرهای مدیریتی مورد نیاز در سامانه پیاده‌سازی شده‌اند یا نه. معیار دوم، درستی رفتار در سناریوهای بحرانی است و مسیرهایی مانند احراز هویت، کنترل موجودی، صورت‌حساب، گزارش‌گیری و نرمال‌سازی پاسخ ارائه‌دهندگان را با تکیه بر آزمون‌های خودکار می‌سنجد. معیار سوم به انسجام معماری و قابلیت توسعه مربوط می‌شود و بررسی می‌کند که آیا تفکیک مسئولیت‌ها، الگوی خط لوله و آداپتورها توسعه‌ی بعدی سامانه را تسهیل می‌کنند. معیار چهارم نیز مرز اعتبار نتایج را روشن می‌کند؛ به این معنا که این پایان‌نامه درستی نرم‌افزاری و کفایت معماری را می‌سنجد و مدعی بنچ‌مارک میدانی گسترده یا یکپارچه‌سازی کامل با همه‌ی سرویس‌های بیرونی نیست.

۶-۱ دستاوردهای پروژه

پروژه‌ی حاضر چند دستاورد مشخص را به‌عنوان خروجی اصلی ارائه می‌دهد. مهم‌ترین آن‌ها یک سامانه‌ی واسط کارکردی در سمت سرور است که درخواست‌های هوش مصنوعی را از سرویس‌گیرندگان دریافت و به چهار ارائه‌دهنده‌ی OpenAI، Gemini، Groq و OpenRouter هدایت می‌کند؛ شواهد ارزیابی این ادعا در فصل پنجم آمده است. در کنار آن، یک رابط برنامه‌نویسی سازگار با OpenAI پیاده‌سازی شده است تا سرویس‌گیرندگان موجود با حداقل تغییر به سامانه متصل شوند.

از نظر درونی، معماری سامانه بر چهارده مرحله‌ی پردازشی مستقل در قالب خط لوله‌ی Laravel استوار است و همین موضوع توسعه و آزمون آن را ساده‌تر می‌کند. همچنین سیستم صورت‌حساب با استفاده از توکن‌ساز واقعی^{۲۰} برای سنجش مصرف، پشتیبانی از صورت‌حساب‌گذاری منابع غیرتوکنی مانند تصویر و صوت، و مدیریت کیف پول با تراکنش‌های ایمن تکمیل شده است. Playground مبتنی بر Vue.js برای آزمایش تعاملی نقاط پایانی و مجموعه‌ای از آزمون‌های واحد و ویژگی نیز این خروجی را از حالت صرفاً نمایشی خارج کرده و به یک کدپایه‌ی قابل ارزیابی تبدیل کرده‌اند.

^{۲۰}Real BPE Tokenizer

۷-۱ تغییرات نسبت به پروپوزال

در پروپوزال^{۲۱} اولیه‌ی این پروژه، بسته‌ی فناوری پیش‌بینی‌شده شامل زبان Python، چارچوب‌های FastAPI یا Django، و پایگاه داده‌ی PostgreSQL بود. در مرحله‌ی اجرا، این بسته‌ی فناوری تغییر یافت و پیاده‌سازی نهایی با فناوری‌های Laravel بر پایه‌ی PHP در سمت سرور، SQL Server برای ذخیره‌سازی داده، Redis برای موتور کش^{۲۲} و مدیریت صف‌های پس‌زمینه، و Vue.js برای پیاده‌سازی میدان بازی^{۲۳} انجام شد.

اهداف اصلی پروژه - شامل یکپارچه‌سازی چند ارائه‌دهنده‌ی هوش مصنوعی، سیستم احراز هویت، مدیریت کلیدهای API، کیف پول، اشتراک و گزارش‌دهی - بدون تغییر باقی ماندند و کاملاً پیاده‌سازی شدند.

۸-۱ ساختار پایان‌نامه

مابقی این پایان‌نامه به این صورت سازماندهی شده است: در فصل دوم، مفاهیم پایه‌ی سامانه‌های واسط و رابط‌های برنامه‌نویسی هوش مصنوعی بررسی می‌شوند و سپس نمونه‌های مشابهی مانند Groq، OpenRouter، Anyscale Endpoints و OpenAI تحلیل و مقایسه می‌شوند. فصل سوم به طراحی سامانه اختصاص دارد و معماری کلی، مدل داده، طراحی API، الگوی خط لوله، استراتژی کش و سیستم صورت‌حساب را شرح می‌دهد. در فصل چهارم، جزئیات پیاده‌سازی ماژول‌های اصلی از جمله خط لوله‌ی سامانه‌ی واسط، آداپتورهای ارائه‌دهندگان، احراز هویت، مدیریت کلیدهای API و صورت‌حساب ارائه می‌شود. فصل پنجم نتایج آزمون‌های واحد و ویژگی، ارزیابی عملکرد و مقایسه با اهداف اولیه را جمع‌بندی می‌کند و فصل ششم به نتیجه‌گیری و پیشنهاد کارهای آتی می‌پردازد.

^{۲۱} Proposal

^{۲۲} Cache Engine

^{۲۳} Playground

فصل دوم

مروری بر پیشینه

۱-۲ مقدمه

در این فصل، پیشینه‌ی نظری و عملی مسئله‌ی یکپارچه‌سازی سرویس‌های هوش مصنوعی بررسی می‌شود. مسئله‌ی اصلی این پروژه صرفاً دسترسی به یک مدل هوشمند نیست، بلکه مدیریت هم‌زمان چند ارائه‌دهنده، یکسان‌سازی رابط‌های برنامه‌نویسی، کنترل دسترسی، صورت‌حساب‌گذاری و گزارش‌گیری در یک بستر واحد است. به همین دلیل، مرور پیشینه باید هم مفاهیم معماری نرم‌افزار و هم نمونه‌سامانه‌های عملی موجود در بازار را در بر گیرد.

در سال‌های اخیر، رابط‌های برنامه‌نویسی کاربردی^۱ به روش غالب دسترسی به مدل‌های هوش مصنوعی تبدیل شده‌اند. از سوی دیگر، رشد سریع مدل‌های زبانی بزرگ باعث شده است که ارائه‌دهندگان متعدد، هر یک با قرارداد ارتباطی، قالب داده، سیاست مالی و محدودیت‌های اجرایی خاص خود، وارد این حوزه شوند. نتیجه آن است که مسئله‌ی یکپارچه‌سازی^۲ اکنون به اندازه‌ی خود مدل‌ها اهمیت دارد.

۲-۲ مفاهیم پایه

۱-۲-۲ سامانه‌ی واسط API

سامانه‌ی واسط^۳ در معماری نرم‌افزار، لایه‌ای است که بین مصرف‌کنندگان سرویس و سامانه‌های پشتیبان قرار می‌گیرد و مسئولیت‌هایی مانند احراز هویت، اعتبارسنجی درخواست، محدودسازی نرخ، راهبری، ثبت گزارش و یکسان‌سازی پاسخ را بر عهده می‌گیرد. در معماری‌های وب مبتنی بر سبک انتقال حالت

^۱ Application Programming Interface (API)

^۲ Integration

^۳ Gateway

بازنمایی^۴، این لایه نقش مهمی در کاهش وابستگی مشتری به جزئیات داخلی سرویس‌های پشت صحنه دارد [۵].

در مسئله‌ی حاضر، سامانه‌ی واسط به جای آن که تنها یک Reverse Proxy باشد، یک لایه‌ی منطقی کامل است. این لایه باید تفاوت میان قالب‌های درخواست و پاسخ ارائه‌دهندگان مختلف را پنهان کند، دسترسی کاربران را مدیریت نماید، وضعیت مالی را کنترل کند و گزارش‌های قابل استفاده برای کاربر و مدیر سامانه تولید نماید. بنابراین، سامانه‌ی واسط مورد نظر این پروژه ترکیبی از وظایف API Gateway، Billing Gateway و Integration Layer است.

۲-۲-۲ معماری خدمات‌گرا و ریزسرویس

در معماری‌های خدمات‌گرا و ریزسرویسی، تفکیک مسئولیت‌ها و مرزبندی روشن اجزای سامانه، یکی از اصول کلیدی برای مقیاس‌پذیری و نگهداشت‌پذیری است. به جای آن که منطق احراز هویت، مسيردهی، صورت‌حساب و گزارش‌گیری در یک لایه‌ی یکپارچه و فشرده پیاده‌سازی شوند، هر حوزه به صورت ماژول مستقل طراحی می‌گردد [۱۲، ۱۶]. مزیت چنین رویکردی آن است که تغییر در یک حوزه اثر جانبی کمتری بر سایر حوزه‌ها می‌گذارد، آزمون‌پذیری سامانه افزایش می‌یابد زیرا هر بخش مرز رفتاری روشن‌تری پیدا می‌کند، و افزودن قابلیت‌های جدیدی مانند راهبری هوشمند یا مدار قطعه بدون بازنویسی ساختار اصلی امکان‌پذیر می‌شود.

هرچند پروژه‌ی حاضر در قالب یک سامانه‌ی متمرکز Laravel پیاده‌سازی شده است، اما در طراحی داخلی آن از همین اصل تفکیک مسئولیت استفاده شده و لایه‌های دامن، سرویس، مخزن، منبع و خط لوله از یکدیگر جدا شده‌اند. این موضوع باعث می‌شود ساختار سامانه، از منظر معماری، رفتاری نزدیک به یک معماری خدمات‌گرا داشته باشد.

۳-۲-۲ مدل‌های زبانی بزرگ و سرویس‌های هوش مصنوعی

تحول اصلی در سرویس‌های هوش مصنوعی معاصر، با فراگیر شدن معماری ترنسفورمر^۵ و مدل‌های زبانی بزرگ رخ داد [۱۷]. این مدل‌ها به واسطه‌ی آموزش روی داده‌های وسیع و پشتیبانی از الگوهای چندوجهی^۶، امروز در کاربردهایی مانند گفتگو، خلاصه‌سازی، جاسازی متن، تولید تصویر و تبدیل گفتار

^۴ Representational State Transfer (REST)

^۵ Transformer

^۶ Multimodal

به متن به کار می‌روند.

با وجود شباهت کارکردی، سرویس‌های مبتنی بر این مدل‌ها از نظر قرارداد فنی تفاوت‌های محسوسی دارند. برای نمونه، OpenAI مجموعه‌ای از نقاط پایانی نسبتاً استاندارد برای گفتگو، responses، جاسازی متن، تصویر و صوت ارائه می‌دهد [۱۳]. در مقابل، Gemini از API بومی خود استفاده می‌کند و نقاط پایانی اصلی آن generateContent و embedContent هستند [۶]. در نتیجه، قالب پیام‌ها و فراساختار مصرف در این سرویس متفاوت است. این تفاوت‌ها نشان می‌دهند که یک سامانه‌ی واسطه مؤثر باید هم از سازگاری بیرونی^۷ و هم از نگاشت درونی^۸ پشتیبانی کند.

۴-۲-۲ رابط‌های سازگار با OpenAI

در عمل، رابط OpenAI به یکی از مراجع غیررسمی این حوزه تبدیل شده است. بسیاری از ابزارهای مشتری، کتابخانه‌ها و حتی برخی ارائه‌دهندگان دیگر، برای کاهش هزینه‌ی مهاجرت مشتریان، خود را با این قرارداد بیرونی سازگار می‌کنند [۱۳، ۸، ۱۴]. چنین روندی از یک سو هزینه‌ی تغییر ارائه‌دهنده را کاهش می‌دهد، اما از سوی دیگر تفاوت در قابلیت‌ها، مدل‌ها و فیلدهای مصرف را از بین نمی‌برد؛ بنابراین این ناهمگنی باید در لایه‌ی واسطه مدیریت شود.

بر همین اساس، استفاده از یک رابط بیرونی سازگار با OpenAI در این پروژه یک تصمیم معماری راهبردی بوده است؛ زیرا هم برای توسعه‌دهندگان آشناست و هم امکان پشتیبانی از چند ارائه‌دهنده را با حداقل تغییر در سمت مشتری فراهم می‌کند.

۳-۲ سیستم‌های مشابه

۱-۳-۲ OpenAI

OpenAI یکی از مهم‌ترین بازیگران این حوزه است و مستندات آن عملاً به مرجع طراحی بسیاری از مشتریان و سامانه‌های واسطه تبدیل شده‌اند [۱۳]. مزیت اصلی این بستر، انسجام رابط برنامه‌نویسی، پوشش چند نوع خدمت هوش مصنوعی و فراگیری ابزارهای پیرامونی آن است.

با این حال، OpenAI ذاتاً یک ارائه‌دهنده‌ی منفرد است، نه یک لایه‌ی تجمیع‌کننده‌ی چند ارائه‌دهنده. بنابراین مسائلی مانند مهاجرت میان ارائه‌دهندگان، مقایسه‌ی قیمت‌ها، اعمال سیاست‌های مالی داخلی

External Compatibility^۷

Internal Mapping^۸

سازمان و مدیریت محلی کاربران، خارج از دامنه‌ی اصلی آن قرار دارند.

Gemini ۲-۳-۲

Gemini به‌عنوان بستر بومی Google DeepMind، قابلیت‌های زبانی و چندوجهی قدرتمندی ارائه می‌کند، اما از نظر شکل API با الگوی OpenAI یکسان نیست [۶]. درخواست‌ها در این بستر معمولاً بر پایه‌ی ساختار contents/parts و پاسخ‌ها با قالبی متفاوت از فیلدهای متعارف OpenAI ارائه می‌شوند. از منظر این پایان‌نامه، اهمیت Gemini در آن است که نشان می‌دهد یکپارچه‌سازی چند ارائه‌دهنده فقط مسئله‌ی تغییر Base URL نیست؛ بلکه در برخی موارد به نگاشت ساختاری درخواست، پاسخ و متاداده‌ی مصرف نیاز دارد. بنابراین، پشتیبانی از Gemini محک خوبی برای ارزیابی انعطاف‌پذیری معماری سامانه‌ی واسط است.

Groq ۳-۳-۲

Groq بستری است که با تأکید بر سرعت بالای استنتاج و با ارائه‌ی API سازگار با OpenAI شناخته می‌شود [۸]. مزیت اصلی آن برای توسعه‌دهنده این است که مهاجرت از مشتریان مبتنی بر OpenAI به‌سادگی انجام می‌شود و در بسیاری از موارد، تغییر اصلی فقط در model, provider و اطلاعات احراز هویت خلاصه می‌شود.

با وجود این سازگاری، Groq نیز یک تجمیع‌کننده‌ی چندارائه‌دهنده نیست و منطق مدیریت کاربران، کیف پول، اشتراک و گزارش‌های سازمانی را بر عهده ندارد. از این رو، شباهت آن به پروژه‌ی حاضر بیشتر در لایه‌ی قرارداد فنی API است تا در حوزه‌ی مدیریت سرویس.

OpenRouter ۴-۳-۲

OpenRouter از نزدیک‌ترین نمونه‌های تجاری به ایده‌ی این پروژه است؛ زیرا یک رابط نسبتاً واحد برای دسترسی به چندین ارائه‌دهنده و مدل مختلف فراهم می‌کند [۱۴]. این بستر نشان می‌دهد که مفهوم تجمیع هوش مصنوعی^۹ در عمل نیز دارای تقاضای واقعی است.

با این حال، تمرکز اصلی OpenRouter بر عرضه‌ی یک API عمومی برای توسعه‌دهندگان است، نه بر مدیریت داخلی یک سازمان. در نتیجه، ویژگی‌هایی مانند احراز هویت مبتنی بر OTP، کیف پول

^۹ AI Aggregation

کاربرمحور، نقش‌ها و سطوح دسترسی، ثبت ممیزی عملیاتی و گزارش‌های مالی سفارشی، در سطحی که برای یک سامانه‌ی سازمانی لازم است، در مرکز طراحی آن قرار ندارند.

۵-۳-۲ Anyscale Endpoints

Anyscale Endpoints نیز بستری برای ارائه و میزبانی مدل‌ها در قالب خدمات قابل فراخوانی است [۱]. نقطه‌ی قوت این بستر بیشتر در حوزه‌ی استقرار، مقیاس‌پذیری عملیاتی و خدمت‌رسانی مدل‌ها است. در نتیجه، تمرکز آن با سامانه‌ی حاضر تفاوت دارد: Anyscale بیشتر مسئله‌ی خدمت‌رسانی مدل^{۱۰} را حل می‌کند، در حالی که پروژه‌ی حاضر مسئله‌ی یکپارچه‌سازی دسترسی و مدیریت مصرف^{۱۱} هدف قرار داده است.

۴-۲ شکاف پژوهشی و معیارهای مقایسه

بررسی پیشینه نشان می‌دهد که در این مسئله با دو دسته منبع روبه‌رو هستیم. دسته‌ی نخست، منابع معماری نرم‌افزار هستند که اصولی مانند REST، تفکیک مسئولیت، و الگوهای خدمات‌گرا را توضیح می‌دهند [۵، ۱۲، ۱۶]. دسته‌ی دوم، مستندات رسمی ارائه‌دهندگان هستند که قراردادهای واقعی API، مدل‌های قیمت‌گذاری و قابلیت‌های اجرایی را مشخص می‌کنند [۱۳، ۶، ۸، ۱۴، ۱]. از آن‌جا که موضوع این پایان‌نامه یک مسئله‌ی پیاده‌سازی‌محور و وابسته به قراردادهای واقعی سرویس‌ها است، استفاده از مستندات رسمی در کنار منابع معماری یک انتخاب روش‌مند است، نه صرفاً یک اتکای توصیفی. برای آن‌که مقایسه‌ی سامانه‌های مشابه صرفاً فهرست‌وار نباشد، در این پایان‌نامه پنج معیار تحلیلی برای سنجش آن‌ها در نظر گرفته می‌شود. این معیارها عبارت‌اند از سازگاری بیرونی، یعنی میزان نزدیکی رابط سامانه به قرارداد OpenAI و هزینه‌ی مهاجرت مشتریان؛ پوشش چندارائه‌دهنده، یعنی این‌که سامانه واقعاً چند ارائه‌دهنده را تجمیع می‌کند یا صرفاً بستری برای یک ارائه‌دهنده است؛ عمق نداشت درونی، یعنی توانایی سامانه در پنهان‌سازی تفاوت‌های ساختاری میان ارائه‌دهندگان سازگار و ناسازگار؛ پشتیبانی مدیریتی و مالی، یعنی وجود قابلیت‌هایی مانند احراز هویت، مدیریت کلید، کنترل دسترسی، کیف پول، اشتراک و گزارش‌گیری؛ و در نهایت جهت‌گیری استقرار، یعنی این‌که سامانه برای مصرف عمومی توسعه‌دهنده، میزبانی مدل یا مدیریت مصرف در سطح سازمان طراحی شده است.

^{۱۰}Model Serving

^{۱۱}

بر اساس این معیارها، شکاف اصلی پیشینه را می‌توان چنین صورت‌بندی کرد: سامانه‌های موجود یا روی ارائه‌ی مستقیم مدل تمرکز دارند، یا روی تجمیع عمومی دسترسی؛ اما ترکیب هم‌زمان قرارداد بیرونی آشنا، نگاشت چند ارائه‌دهنده، و کنترل مالی و مدیریتی در سطح سازمان کمتر دیده می‌شود. این شکاف، مبنای طراحی سامانه‌ی پیشنهادی در فصل بعدی است.

۲-۵ مقایسه و جمع‌بندی

مرور سامانه‌های فوق نشان می‌دهد که هر یک بخشی از مسئله را پوشش می‌دهند، اما هیچ‌کدام دقیقاً مجموعه‌ی نیازهای این پروژه را به‌صورت یکجا پاسخ نمی‌دهند. جدول ۱-۲ خلاصه‌ای از این مقایسه را نشان می‌دهد.

جدول ۱-۲: مقایسه‌ی سامانه‌های بررسی‌شده با مسئله‌ی این پایان‌نامه

سامانه	چند ارائه‌دهنده	سازگار با OpenAI	مدیریت داخلی	تمرکز اصلی
OpenAI	خیر	بله	خیر	ارائه‌ی مستقیم سرویس‌های هوش مصنوعی
Gemini	خیر	خیر	خیر	ارائه‌ی بومی مدل‌های چندوجهی
Groq	خیر	بله	خیر	استنتاج سریع با قرارداد سازگار
OpenRouter	بله	بله	نسبی	تجمیع عمومی چند مدل برای توسعه‌دهندگان
Anyscale Endpoints	نسبی	نسبی	خیر	میزبانی و خدمت‌رسانی مدل‌ها
سامانه‌ی این پایان‌نامه	بله	بله	بله	یکپارچه‌سازی فنی، مالی و مدیریتی در سطح سازمان

در نتیجه، شکاف اصلی در پیشینه‌ی موجود را می‌توان چنین خلاصه کرد: سامانه‌ای مورد نیاز است که از یک سو برای توسعه‌دهنده، رابطی آشنا و سازگار با OpenAI ارائه دهد و از سوی دیگر، نیازهای مدیریتی و مالی یک سازمان را نیز پوشش دهد. این شکاف، مبنای طراحی سامانه‌ی پیشنهادی این

پایان نامه است که در فصل بعدی معماری و مدل داده‌ی آن تشریح می‌شود.

فصل سوم

طراحی سیستم

۱-۳ مقدمه

پس از بررسی مسئله و پیشینه، در این فصل طراحی سامانه‌ی پیاده‌سازی شده تشریح می‌شود. هدف از این طراحی، ساخت بستری است که بتواند هم به عنوان نقطه‌ی ورود یکپارچه برای درخواست‌های هوش مصنوعی عمل کند و هم قابلیت‌های مدیریتی، مالی و نظارتی لازم برای بهره‌برداری سازمانی را فراهم آورد.

در طراحی سامانه، سه اصل راهنما در نظر گرفته شده است: نخست، **سازگاری بیرونی** با رابط OpenAI برای کاهش هزینه‌ی مهاجرت مشتریان؛ دوم، **تفکیک مسئولیت‌ها** در لایه‌های داخلی برای افزایش نگهداشت‌پذیری؛ و سوم، **قابلیت کنترل و گزارش‌گیری** برای مدیریت مصرف، هزینه و امنیت.

۲-۳ نیازمندی‌های طراحی

پیش از تشریح معماری، لازم است نیازمندی‌هایی که طراحی باید به آن‌ها پاسخ دهد به‌صورت صریح فهرست شوند. جدول ۱-۳ این نیازمندی‌ها را در دو دسته‌ی کارکردی و غیرکارکردی خلاصه می‌کند.

جدول ۳-۱: نیازمندی‌های اصلی مؤثر بر طراحی سامانه

شناسه	نوع	شرح
F1	کارکردی	ارائه‌ی نقاط پایانی سازگار با OpenAI برای گفتگو، responses، جاسازی متن، تصویر و صوت
F2	کارکردی	پشتیبانی از چند ارائه‌دهنده با امکان نگاشت تفاوت‌های قراردادی در لایه‌ی درونی
F3	کارکردی	احراز هویت کاربر، مدیریت مشتریان API و کلیدهای برنامه‌ای
F4	کارکردی	کنترل مالی شامل کیف پول، اشتراک، فاکتور و ثبت دقیق مصرف
F5	کارکردی	گزارش‌گیری از مصرف، تراکنش‌ها و عملیات حساس
NF1	غیرکارکردی	کاهش وابستگی مشتری به ارائه‌دهنده و تسهیل مهاجرت میان سرویس‌ها
NF2	غیرکارکردی	تفکیک مسئولیت، آزمون‌پذیری و قابلیت توسعه‌ی تدریجی
NF3	غیرکارکردی	کنترل امنیتی شامل محدودسازی نرخ، فهرست سفید IP و ثبت ممیزی
NF4	غیرکارکردی	خارج کردن کارهای غیر بحرانی از مسیر اصلی درخواست با تکیه بر کش و صف

چند نیازمندی نیز آگاهانه از دامنه‌ی نسخه‌ی فعلی خارج شده‌اند. برای نمونه، انتخاب هوشمند ارائه‌دهنده، اتصال OTP به سرویس واقعی پیامک یا ایمیل و یکپارچه‌سازی با درگاه پرداخت بیرونی به‌عنوان جهت توسعه‌ی آتی نگه داشته شده‌اند. بنابراین طراحی فعلی باید به‌گونه‌ای باشد که این قابلیت‌ها را در آینده بدون بازنویسی گسترده بپذیرد.

۳-۳ معماری کلی سیستم

سامانه از چند زیرسامانه‌ی مکمل تشکیل شده است که هر کدام مسئول بخش مشخصی از چرخه‌ی خدمت‌رسانی هستند. نمای منطقی این اجزا در جدول ۲-۳ خلاصه شده است.

جدول ۲-۳: اجزای اصلی معماری سامانه

جزء	وظیفه
REST API Layer	ارائه‌ی نقاط پایانی نسخه‌بندی‌شده برای احراز هویت، مدیریت کاربران، کلیدهای API، کیف پول، اشتراک، گزارش و درخواست‌های هوش مصنوعی
Gateway Core	اعتبارسنجی، انتخاب ارائه‌دهنده و مدل، تخمین مصرف، فراخوانی ارائه‌دهنده، یکسان‌سازی پاسخ و ثبت مصرف
Billing Core	مدیریت کیف پول، تراکنش‌ها، اشتراک‌ها، فاکتورها و اعمال هزینه‌ی هر درخواست
Identity & Access	احراز هویت مبتنی بر OTP، مدیریت نقش‌ها، مشتریان API و کلیدها
Reporting	تولید گزارش مصرف، دفترکل کیف پول، گزارش فاکتورها و انبارسازی تجمیعی
SQL Server	نگهداری داده‌های پایدار شامل کاربران، مدل‌ها، مصرف، تراکنش‌ها و گزارش‌ها
Redis	کش داده‌های پرتکرار، شمارنده‌های محدودسازی نرخ و صف کارهای پس‌زمینه
Queue Workers	اجرای غیرهمگام ارسال OTP، تولید PDF فاکتور، تجمیع روزانه و ارسال گزارش خارجی
Playground UI	محیط آزمایشی برای ارسال تعاملی درخواست و مشاهده‌ی پاسخ سامانه

از دید تعامل، سامانه سه مسیر اصلی دارد. در مسیر مدیریتی، کاربر انسانی پس از احراز هویت با OTP به داشبورد و نقاط پایانی مدیریتی دسترسی پیدا می‌کند. در مسیر برنامه‌ای، نرم‌افزارهای مشتری با کلید API درخواست‌های هوش مصنوعی را به نقاط پایانی سازگار با OpenAI می‌فرستند. مسیر سوم نیز به کارهای پس‌زمینه اختصاص دارد؛ کارهایی مانند ارسال گزارش خارجی و تولید PDF که زمان‌بر یا غیر بحرانی هستند از طریق صف اجرا می‌شوند.

۳-۳-۱ جریان درخواست

جریان منطقی یک درخواست هوش مصنوعی در سامانه از دریافت آن در نقطه‌ی پایانی `/api/v1/ai/...` آغاز می‌شود. سپس کلید API و وضعیت کاربر مرتبط استخراج و اعتبارسنجی می‌شود، محدودیت نرخ و مجاز بودن آدرس IP بررسی می‌گردد و ارائه‌دهنده و مدل بر اساس مقادیر صریح `provider` و `model` تعیین می‌شوند. پس از آن، سامانه پیش از تماس با ارائه‌دهنده مصرف و هزینه‌ی احتمالی را تخمین می‌زند و وضعیت اشتراک یا موجودی کیف پول را کنترل می‌کند. اگر درخواست مجاز باشد، فراخوانی از طریق آداپتور مناسب انجام می‌شود، پاسخ یکدست‌سازی می‌گردد، مصرف واقعی از آن استخراج یا با تخمین تلفیق می‌شود، رکوردهای گزارش و مصرف در پایگاه داده ثبت می‌شوند و در پایان هزینه اعمال و در صورت نیاز گزارش خارجی به صورت غیرهمگام ارسال می‌شود. این جریان به گونه‌ای طراحی شده است که خطاهای مالی یا امنیتی پیش از ارسال درخواست به ارائه‌دهنده تشخیص داده شوند و از هزینه‌ی غیرضروری جلوگیری شود.

۳-۴-۲ مدل داده

مدل داده‌ی سامانه بر اساس چهار حوزه‌ی اصلی طراحی شده است: هویت و دسترسی، ارائه‌دهندگان و مدل‌ها، مالی و صورت حساب، و گزارش‌گیری. این تقسیم‌بندی با تفکیک ماژول‌های نرم‌افزار نیز هم‌راستا است.

۳-۴-۱ جداول احراز هویت و دسترسی

هسته‌ی مدیریت کاربران بر جدول `users` استوار است و با جداول `roles`، `permissions` و جداول واسطه مربوطه تکمیل می‌شود. برای ورود کاربر به سامانه، جدول `otp_challenges` نگهدارنده‌ی مقصد، کانال، کد هش شده، زمان انقضا و تعداد دفعات تلاش است.

برای دسترسی برنامه‌ای نیز دو سطح مجزا در نظر گرفته شده است. سطح نخست `api_clients` است که یک برنامه یا مصرف‌کننده‌ی منطقی متعلق به کاربر را نمایش می‌دهد. سطح دوم `api_keys` است که پیشوند کلید، هش کلید، حوزه‌های دسترسی، محدودیت نرخ، فهرست IP‌های مجاز و تاریخ انقضا یا ابطال را نگهداری می‌کند.

این طراحی باعث می‌شود کاربر بتواند برای چند نرم‌افزار مختلف، کلیدهای مجزا با سیاست‌های امنیتی متفاوت ایجاد کند.

۳-۴-۲ جداول ارائه‌دهندگان و مدل‌ها

پیکربندی ارائه‌دهندگان در جدول `providers` و مشخصات مدل‌ها در جدول `provider_models` ذخیره می‌شود. هر ارائه‌دهنده شامل نام، نوع، Base URL، وضعیت، اولویت و پیکربندی محرمانه است. هر مدل نیز با `model\key`، قابلیت‌ها و پیکربندی قیمت‌گذاری توصیف می‌شود.

در کنار این دو جدول، جدول `routing_rules` نیز برای توسعه‌ی آینده در نظر گرفته شده است. هرچند در نسخه‌ی فعلی، انتخاب ارائه‌دهنده و مدل به صورت صریح توسط مشتری انجام می‌شود، وجود این جدول امکان پیاده‌سازی راهبردهای پیشرفته‌تر را در آینده فراهم می‌کند.

۳-۴-۳ جداول صورت‌حساب

بخش مالی سامانه از چند جدول مکمل تشکیل شده است. جدول `wallets` موجودی اصلی هر کاربر و واحد پول را نگه می‌دارد، `wallet_transactions` دفترکل تراکنش‌های بدهکار و بستانکار است، `subscription_plans` طرح‌های اشتراک و قیمت و امکانات آن‌ها را تعریف می‌کند، `subscriptions` وضعیت اشتراک فعال کاربر و بازه‌ی اعتبار آن را ثبت می‌نماید و در نهایت جداول `invoices` و `invoice_items` برای نگهداری فاکتورها و اقلام هر فاکتور به کار می‌روند.

این ساختار دو الگوی مالی را به صورت هم‌زمان پشتیبانی می‌کند: مصرف بر اساس کیف پول و مصرف مبتنی بر اشتراک.

۳-۴-۴ جداول گزارش مصرف

هر درخواست عبوری از سامانه در صورت فعال بودن ثبت گزارش، در جدول `gateway_requests` ذخیره می‌شود. سپس اقلام مصرف قابل‌صورتحساب در جدول `usage_records` ثبت می‌گردند. برای کاهش

هزینه‌ی گزارش‌گیری بلندمدت، خلاصه‌های روزانه در جدول `daily_usage_rollups` تولید می‌شود. دو جدول مکمل دیگر نیز در این حوزه وجود دارند: `audit_logs` برای ثبت عملیات حساس، مانند ایجاد یا ابطال کلید، و `provider_health_checks` برای پایش سلامت ارائه‌دهندگان.

۳-۵ طراحی API

طراحی API بر دو اصل استوار است: نسخه‌بندی شفاف و تفکیک نقاط پایانی مدیریتی از نقاط پایانی هوش مصنوعی. همه‌ی مسیرها زیر پیشوند `/api/v1` قرار گرفته‌اند تا توسعه‌ی نسخه‌های بعدی بدون شکستن سازگاری ممکن باشد.

۳-۵-۱ نقاط پایانی احراز هویت و مدیریتی

نقاط پایانی مدیریتی از توکن نشست Sanctum استفاده می‌کنند [۹]. این مسیرها در عمل چند گروه را پوشش می‌دهند: مسیرهای احراز هویت مانند `/auth/otp/start` و `/auth/otp/verify`، مسیر پروفایل کاربر در `/me`، مسیرهای مدیریت مشتری و کلید در `/api-clients` و `/api-keys/...`، مسیرهای مالی مانند `/wallet`، `/plans`، `/subscriptions` و `/invoices`، مسیرهای گزارش‌گیری مانند `/reports/usage`، `/reports/wallet-ledger` و `/reports/invoices`، و در نهایت مسیرهای مدیریت ارائه‌دهندگان در `/providers` و `/providers/{provider}/models`. علاوه بر این، دریافت PDF فاکتور با مسیر `/invoices/{invoice}/pdf` و از طریق پیوند امضاشده^۱ طراحی شده است تا دسترسی کنترل‌شده و موقت به فایل فراهم شود.

۳-۵-۲ نقاط پایانی هوش مصنوعی

برای بیشینه کردن سازگاری با مشتریان موجود، شش نقطه‌ی پایانی اصلی مطابق قرارداد بیرونی OpenAI در نظر گرفته شده است: `/ai/responses`، `/ai/chat/completions`، `/ai/embeddings`، `/ai/images/generations`، `/ai/audio/transcriptions` و `/ai/audio/speech`. همه‌ی این درخواست‌ها با کلید API احراز هویت می‌شوند و به‌صورت صریح دارای دو فیلد `provider` و `model` هستند. این تصمیم طراحی دو مزیت دارد: نخست، از ابهام در قیمت‌گذاری جلوگیری می‌کند؛ دوم، مسئولیت انتخاب ارائه‌دهنده را برای مشتری شفاف می‌سازد.

^۱ Signed URL

۳-۶ الگوی خط لوله

هسته‌ی طراحی سامانه بر الگوی خط لوله^۲ استوار است. در این الگو، هر مرحله یک مسئولیت مشخص دارد و ورودی و خروجی آن در قالب یک زمینه‌ی درخواست^۳ مشترک حمل می‌شود. مزایای اصلی این الگو در پروژه‌ی حاضر روشن است: جریان پردازش خواناتر می‌شود و نگرانی‌های مختلف از هم جدا می‌مانند، هر مرحله را می‌توان به‌صورت مستقل آزمود، افزودن یا حذف مرحله‌ها بدون دستکاری گسترده‌ی کد ممکن می‌شود و مدیریت خطا و توقف زودهنگام در صورت بروز مشکل ساده‌تر خواهد بود. در نسخه‌ی فعلی، خط لوله‌ی درگاه شامل چهارده مرحله است:

۱. ResolveApiKeyPipe

۲. EnforceIpAllowlistPipe

۳. ValidateGatewayPayloadPipe

۴. RateLimitPipe

۵. SelectProviderPipe

۶. ValidateProviderSelectionPipe

۷. EstimateUsagePipe

۸. CheckSubscriptionOrWalletPipe

۹. DispatchProviderRequestPipe

۱۰. NormalizeProviderResponsePipe

۱۱. MeterUsagePipe

۱۲. PersistLogsPipe

۱۳. ChargeUsagePipe

^۲ Pipeline Pattern

^۳ Request Context

۱۴. DispatchExternalLogsPipe

ترتیب این مراحل عمدی است. برای نمونه، تخمین مصرف باید پیش از بررسی کیف پول انجام شود، و ثبت گزارش نیز باید پس از استخراج مصرف واقعی انجام گیرد.

۳-۷ استراتژی کش

در طراحی سامانه، کش با راهبرد کش کنار^۴ در نظر گرفته شده است. در این الگو، داده ابتدا از کش خوانده می‌شود و در صورت نبود، از پایگاه داده بازیابی و سپس در کش ذخیره می‌گردد. این راهبرد برای داده‌های کم‌تغییر و پرتکرار مناسب است.

مهم‌ترین اقلامی که در طراحی سامانه برای کش در نظر گرفته شده‌اند شامل فهرست طرح‌های فعال با کلید `plans:active`، فهرست ارائه‌دهندگان با کلید `providers:all`، پیکربندی هر ارائه‌دهنده با الگوی `providers:config:providerName`، مدل‌های هر ارائه‌دهنده با الگوی `provider_models:list:providerId` و پروفایل کاربر با الگوی `user:profile:userId` هستند.

زمان اعتبار هر گروه از طریق پیکربندی مرکزی تعیین می‌شود تا بتوان سیاست کش را بدون تغییر در منطق سرویس‌ها تنظیم کرد. استفاده از Redis به عنوان بستر کش، علاوه بر سرعت مناسب، امکان هم‌زیستی طبیعی کش، محدودسازی نرخ و صف را نیز فراهم می‌کند [۱۵].

۳-۸ طراحی سیستم صورت حساب

صورت حساب‌گذاری در سامانه بر پایه‌ی مدل‌های قیمت‌گذاری ذخیره‌شده در فیلد `pricing_config` از جدول `provider_models` طراحی شده است. این تصمیم باعث می‌شود که منطق قیمت، به‌جای پراکندگی در کد، به‌صورت داده‌محور و قابل تغییر مدیریت شود.

برای هر درخواست، ابتدا یک تخمین از مصرف انجام می‌شود تا امکان پیش‌بررسی موجودی وجود داشته باشد. سپس پس از دریافت پاسخ، مصرف واقعی از پاسخ ارائه‌دهنده استخراج یا با تخمین پیشین تلفیق می‌شود. هزینه‌ی نهایی هر درخواست از رابطه‌ی زیر به دست می‌آید:

$$C = \sum_{i=1}^n q_i \times p_i$$

^۴Cache-Aside

که در آن q_i مقدار هر معیار مصرف و p_i هزینه‌ی واحد همان معیار است. این معیارها می‌توانند شامل توکن ورودی، توکن خروجی، تعداد تصویر، ثانیه‌ی صوت و تعداد نویسه‌های ورودی برای TTS باشند. در طراحی مالی، دو مسیر هم‌زمان پشتیبانی می‌شوند. اگر کاربر اشتراک فعال با اعتبار مشمول مصرف داشته باشد، درخواست بدون برداشت مستقیم از کیف پول عبور می‌کند. در غیر این صورت، هزینه از موجودی کیف پول کسر می‌شود و تراکنش متناظر در دفترکل ثبت می‌گردد. فاکتورها نیز به‌عنوان نمایش رسمی اقلام هزینه طراحی شده‌اند و اقلام آن‌ها می‌توانند هم مصرف جاری و هم هزینه‌های اشتراک را پوشش دهند. این طراحی، امکان گزارش‌گیری مالی و تولید PDF رسمی برای هر دوره را فراهم می‌کند.

فصل چهارم

پیاده‌سازی

۴-۱ مقدمه

در این فصل، نحوه‌ی تبدیل طرح مفهومی فصل قبل به یک سامانه‌ی اجرایی توضیح داده می‌شود. پیاده‌سازی انجام‌شده صرفاً یک نمونه‌ی آزمایشی نیست؛ بلکه یک کدپایه‌ی عملیاتی است که لایه‌های مدیریتی، مالی و درگاهی را در کنار هم ارائه می‌کند. در این پیاده‌سازی، تلاش شده است که هم از امکانات آماده‌ی چارچوب Laravel استفاده شود و هم منطق اختصاصی پروژه در لایه‌های مستقل و آزمون‌پذیر قرار گیرد.

۴-۲ فناوری‌های استفاده‌شده

۴-۲-۱ چارچوب Laravel

هسته‌ی سمت سرور سامانه با چارچوب Laravel پیاده‌سازی شده است [۱۰]. انتخاب این چارچوب به این دلیل انجام شد که هم از معماری لایه‌ای و الگوهای متداول توسعه به‌خوبی پشتیبانی می‌کند، هم ابزارهای استاندارد برای اعتبارسنجی، صف، کش، رویداد و احراز هویت در اختیار می‌گذارد. افزون بر آن، پشتیبانی داخلی از Pipeline با نیاز اصلی پروژه هم‌راستا است و سازگاری آن با SQL Server و Redis در محیط ویندوز، پیاده‌سازی عملیاتی سامانه را ساده‌تر کرده است.

افزون بر این، استفاده از Laravel Sanctum برای نقاط پایانی مدیریتی، احراز هویت نشست‌محور کاربر را ساده کرده است [۹].

۴-۲-۲ پایگاه داده‌ی SQL Server

ذخیره‌سازی داده‌های پایدار با SQL Server انجام شده است [۱۱]. این پایگاه داده محل نگهداری اطلاعات کاربران، کلیدها، ارائه‌دهندگان، مدل‌ها، کیف پول، اشتراک‌ها، فاکتورها، گزارش‌ها و لاگ‌های ممیزی است. استفاده از تراکنش‌های پایگاه داده برای عملیات مالی، مانند شارژ و برداشت از کیف پول، یکی از الزامات اجرایی این بخش بوده است.

۴-۲-۳ Redis

Redis در این پروژه سه نقش اصلی دارد [۱۵]. از یک سو برای کش داده‌های پرتکرار مانند طرح‌ها، ارائه‌دهندگان و پروفایل کاربر به کار می‌رود، از سوی دیگر شمارنده‌های محدودسازی نرخ برای OTP و درگاه هوش مصنوعی را نگه می‌دارد، و در نهایت بستر اجرای صف کارهای پس‌زمینه‌ای مانند ارسال OTP و ارسال گزارش خارجی را فراهم می‌کند. ترکیب این سه نقش در یک فناوری واحد، پیچیدگی عملیاتی سامانه را کاهش داده و هماهنگی میان بخش‌های مختلف را بیشتر کرده است.

۴-۲-۴ Vue.js

برای پیاده‌سازی میدان بازی^۱ از Vue.js و ابزار Vite استفاده شده است [۴]. این رابط کاربری وظیفه‌ی اصلی سامانه را بر عهده ندارد، اما برای آزمون دستی نقاط پایانی و مشاهده‌ی سریع رفتار درگاه بسیار مفید است. رابط مذکور امکان انتخاب نقطه‌ی پایانی، ارائه‌دهنده، مدل، کلید API و محتوای JSON را برای کاربر فراهم می‌کند.

۴-۳ ساختار پوشه‌بندی پروژه

کدپایه به صورت حوزه‌محور و لایه‌محور سازمان‌دهی شده است. مهم‌ترین شاخه‌های پروژه به گونه‌ای انتخاب شده‌اند که هر حوزه جای مشخص خود را داشته باشد. برای مثال، app/Domains منطق دامنه‌های اصلی مانند درگاه، احراز هویت و ارائه‌دهندگان را در خود جای داده است؛ app/Services سرویس‌های سطح کاربرد برای مالی، گزارش، کاربران، کش و مدیریت ارائه‌دهندگان را نگه می‌دارد؛

^۱ Playground

app/Repositories لایه‌ی دسترسی به داده را از منطق تجاری جدا می‌کند؛ app/Http/Controllers و app/Http/Resources به ترتیب وظیفه‌ی کنترل ورودی و شکل‌دهی پاسخ‌های نسخه‌بندی‌شده‌ی API را بر عهده دارند؛ app/Pipelines محل خط لوله‌های درگاه و احراز هویت است؛ app/Jobs کارهای صف‌شونده‌ای مانند ارسال OTP، تولید PDF فاکتور و ارسال گزارش خارجی را نگه می‌دارد؛ و در نهایت resources/js/playground به کد رابط کاربری میدان بازی اختصاص یافته است.

این سازمان‌دهی باعث می‌شود که کدهای مرتبط با یک حوزه‌ی مسئله در کنار هم قرار گیرند و در عین حال، مسئولیت‌های اجرایی مختلف از هم جدا بمانند.

۴-۴ پیاده‌سازی خط لوله‌ی سامانه‌ی واسط

هسته‌ی اجرایی سامانه در سرویس GatewayService قرار دارد. این سرویس یک شیء زمینه‌ی درخواست^۲ ایجاد می‌کند و آن را از چهارده مرحله‌ی خط لوله عبور می‌دهد. این مراحل از نظر منطقی به چند بخش قابل فهم تقسیم می‌شوند. در ابتدای مسیر، ResolveApiKeyPipe و EnforceIpAllowlistPipe هویت و مجاز بودن مبدأ را بررسی می‌کنند. پس از آن، مرحله‌های اعتبارسنجی داده و محدودسازی نرخ وارد عمل می‌شوند. در گام بعد، SelectProviderPipe و ValidateProviderSelectionPipe ارائه‌دهنده و مدل را بارگذاری و صحت آن‌ها را کنترل می‌کنند. سپس سامانه در دو مرحله‌ی مجزا مصرف تقریبی را برآورد کرده و وضعیت مالی درخواست را پیش از فراخوانی بیرونی می‌سنجد.

پس از تماس با ارائه‌دهنده، پاسخ در یک گام یکدست‌سازی می‌شود و در انتهای مسیر، مراحل اندازه‌گیری مصرف، ثبت لاگ، اعمال هزینه و ارسال غیرهمگام گزارش ساخت‌یافته به ترتیب اجرا می‌شوند. یکی از مزایای عملی این ساختار آن است که در صورت بروز خطا، هر مرحله می‌تواند زنجیره را متوقف و پاسخ استاندارد تولید کند. به این ترتیب، خطاهای امنیتی، مالی یا اعتبارسنجی قبل از تماس با ارائه‌دهنده تشخیص داده می‌شوند.

^۲ Gateway Request Context

۴-۵ پیاده‌سازی آداپتورهای ارائه‌دهندگان

۴-۵-۱ OpenAI

آداپتور OpenAI-سازگار برای سه ارائه‌دهنده‌ی OpenAI، Groq و OpenRouter استفاده شده است. این آداپتور با نداشت داخلی نقاط پایانی `responses`، `chat/completions`، `embeddings`، `images/generations`، `audio/transcriptions` و `audio/speech` به مسیرهای متناظر، درخواست را به ارائه‌دهنده‌ی بالادستی ارسال می‌کند.

در این بخش چند نکته‌ی اجرایی اهمیت دارد. برای `audio/transcriptions` از ارسال چندبخشی و الحاق فایل استفاده می‌شود، برای `audio/speech` امکان دریافت پاسخ دودویی و بازگرداندن مستقیم آن وجود دارد، و مصرف توکن و در صورت امکان تعداد تصاویر از پاسخ استخراج می‌شود تا در صورت حساب‌گذاری به‌کار رود.

۴-۵-۲ Gemini

پشتیبانی از Gemini در این پروژه از نوع نداشت بومی است، نه صرفاً تغییر `Base URL`. آداپتور `GeminiProviderAdapter` درخواست‌های `chat.completions` و `responses` را به قالب `generateContent` و درخواست‌های `embeddings` را به قالب `embedContent` یا `batchEmbedContents` تبدیل می‌کند.

این آداپتور در عمل چند وظیفه‌ی مهم را انجام می‌دهد: ساختار `messages` یا `input` را به `contents/parts` تبدیل می‌کند، پیام `system` را جدا کرده و به `systemInstruction` تبدیل می‌نماید، فراساختار مصرف Gemini را به فیلد `usage` در قالب سازگار با OpenAI نگاشت می‌کند و خطاهای بومی Gemini را به پاسخ‌های استاندارد درگاه برمی‌گرداند.

در نسخه‌ی فعلی، Gemini برای سه قابلیت گفتگو، `responses` و جاسازی متن پشتیبانی می‌شود و سایر نقاط پایانی چندرسانه‌ای از مسیر OpenAI-سازگار عبور می‌کنند.

۴-۵-۳ OpenRouter و Groq

Groq و OpenRouter با همان آداپتور سازگار با OpenAI پشتیبانی می‌شوند. تفاوت اصلی در این‌جا در سطح پیکربندی نهفته است: `ProviderRegistry` با توجه به نام ارائه‌دهنده، `Base URL` و کلید محرمانه‌ی

مناسب را از پیکربندی یا پایگاه داده بارگذاری می‌کند. این طراحی نشان می‌دهد که برای ارائه‌دهندگان سازگار، توسعه‌ی سامانه بیشتر مبتنی بر داده است تا کدنویسی مجدد.

۴-۶ پیاده‌سازی احراز هویت OTP

منطق احراز هویت کاربر در سرویس OtpService متمرکز شده است. مرحله‌ی شروع چالش^۳ با ایجاد یک OtpContext و عبور آن از یک خط لوله‌ی سه‌مرحله‌ای اجرا می‌شود؛ ThrottleOtpPipe محدودیت نرخ را بر اساس کانال، مقصد و IP اعمال می‌کند، GenerateOtpPipe کد عددی تصادفی را تولید و هش کرده و چالش را ثبت می‌کند، و DispatchOtpPipe ارسال کد را از طریق SendOtpJob به صف می‌سپارد.

در مرحله‌ی تأیید^۴، جدیدترین چالش مربوط به مقصد بازیابی شده و انقضا، تعداد تلاش‌ها و صحت کد بررسی می‌شود. در صورت موفقیت، کاربر موجود بازیابی یا در صورت نیاز به‌طور خودکار ایجاد می‌شود. پیاده‌سازی فعلی SendOtpJob نقش یک مؤلفه‌ی موقت را دارد و عملیات ارسال واقعی را به ثبت در لاگ محدود می‌کند؛ بنابراین منطق احراز هویت کامل است، اما اتصال به ارائه‌دهنده‌ی واقعی پیامک یا ایمیل به‌عنوان کار آتی باقی مانده است.

۴-۷ پیاده‌سازی مدیریت کلیدهای API

مدیریت کلیدهای API در دو لایه پیاده‌سازی شده است. در لایه‌ی دامنه، سرویس ApiKeyService کلید خام را با پیشوند gk_ تولید می‌کند، فقط یک‌بار آن را به کاربر نمایش می‌دهد و سپس نسخه‌ی هش‌شده‌ی آن را ذخیره می‌کند. علاوه بر هش کامل، یک پیشوند کوتاه نیز نگهداری می‌شود تا جست‌وجوی کلید در زمان احراز هویت سریع‌تر انجام شود.

در لایه‌ی کاربرد، عملیات ایجاد، ابطال و چرخش کلید با ثبت لاگ ممیزی همراه است. در سمت دریافت درخواست نیز میان‌افزار AuthenticateApiKey کلید را از سرآیند Authorization: Bearer یا X-API-Key استخراج کرده و وجود و اعتبار کلید، ابطال یا انقضای آن، فعال بودن مشتری و کاربر مرتبط و مجاز بودن IP را در صورت تعریف فهرست سفید کنترل می‌کند.

^۳Start

^۴Verify

این طراحی از ذخیره‌ی رمزهای خام در پایگاه داده جلوگیری کرده و سطح امنیتی مناسبی برای درگاه فراهم می‌کند.

۴-۸ پیاده‌سازی سیستم صورت حساب

سیستم مالی سامانه از چند بخش مکمل تشکیل شده است. نخست، سرویس UsageEstimationService مصرف احتمالی درخواست را برآورد می‌کند. در این سرویس، متن درخواست‌های گفتگو و responses را با استفاده از توکن‌ساز BPE^۵ شمارش می‌کند، در صورت ناشناخته بودن مدل یک Encoding پیش‌فرض به کار می‌گیرد، برای تصویر تعداد تصاویر را از فیلدهایی مانند n استخراج می‌کند و برای صوت مدت زمان یا تعداد نویسه‌ها را به عنوان معیار قیمت‌گذاری غیرتوکنی در نظر می‌گیرد.

سپس UsageMeteringService این مقادیر را با pricing_config مدل انتخاب‌شده ترکیب کرده و رکوردهای هزینه‌ای مانند tokens_in, tokens_out, images, audio_seconds و audio_chars را می‌سازد.

برای برداشت از کیف پول، WalletService از تراکنش پایگاه داده استفاده می‌کند تا افزایش و کاهش موجودی به صورت اتمی انجام شود. سرویس SubscriptionService نیز منطق ایجاد و لغو اشتراک را به همراه برداشت هزینه‌ی طرح از کیف پول پیاده‌سازی می‌کند. در سطح فاکتور، سرویس InvoiceService اقلام را آماده کرده، مجموع و مالیات را محاسبه می‌کند و فایل PDF را از طریق Dompdf تولید می‌نماید. تولید PDF قابل صف‌شدن است و توسط GenerateInvoicePdfJob انجام می‌شود.

۴-۹ پیاده‌سازی گزارش‌دهی و ارسال گزارش خارجی

گزارش‌گیری داخلی و ارسال گزارش خارجی دو مسیر مکمل اما مستقل دارند. در مسیر داخلی، PersistLogsPipe در صورت فعال بودن پیکربندی، یک رکورد در جدول gateway_requests و رکوردهای مصرف متناظر در جدول usage_records ثبت می‌کند. سپس ReportingService با تکیه بر این داده‌ها، گزارش مصرف، دفترکل کیف پول و گزارش فاکتورها را تولید می‌نماید.

برای گزارش‌های تحلیلی بلندمدت، کار AggregateDailyUsageJob مصرف هر روز را بر اساس

^۵Byte Pair Encoding (BPE)

کاربر، کلید، ارائه‌دهنده، مدل و نوع معیار تجمیع می‌کند. این کار هزینه‌ی گزارش‌گیری در بازه‌های زمانی طولانی را کاهش می‌دهد.

در مسیر خارجی، GatewayLogService از زمینه‌ی درخواست یک بار گزارش ساخت‌یافته تولید می‌کند، فیلدهای حساس مانند کلید و توکن را حذف یا پوشانده^۶ می‌کند، در صورت نیاز اندازه‌ی داده را می‌کاهد و سپس از طریق DispatchGatewayLogJob آن را به یکی از مقاصد پشتیبانی‌شده می‌فرستد. انتخاب مقصد توسط LogSinkManager و تنها با تغییر یک مقدار پیکربندی انجام می‌شود. در نسخه‌ی فعلی دو مقصد Grafana Loki [۷] و Better Stack [۲] پیاده‌سازی شده‌اند.

۴-۱۰ داشبورد Playground

رابط Playground در مسیر playground/ در اختیار کاربر قرار دارد و برای آزمون تعاملی نقاط پایانی طراحی شده است. کاربر در این رابط می‌تواند یکی از شش قابلیت اصلی درگاه را انتخاب کند، نام ارائه‌دهنده و مدل و کلید API را وارد کند، JSON درخواست را مستقیماً ویرایش نماید و پاسخ نهایی را در قالب متنی ببیند.

اگرچه این رابط جایگزین یک پنل مدیریتی کامل نیست، اما برای توسعه و اشکال‌زدایی بسیار مفید است؛ زیرا می‌تواند بدون نیاز به ابزار بیرونی، رفتار درگاه و سازگاری پاسخ‌ها را نشان دهد. در کنار این رابط، مستندات OpenAPI نیز با استفاده از L5-Swagger برای پروژه تولید شده است [۳]. ترکیب مستندات ماشینی و محیط آزمون تعاملی، قابلیت استفاده و نگهداری سامانه را بهبود داده است.

فصل پنجم

ارزیابی

۵-۱ مقدمه

هدف از ارزیابی این پروژه، سنجش میزان انطباق پیاده‌سازی با اهداف تعریف‌شده در فصل اول است. از آن‌جا که سامانه‌ی حاضر یک محصول زیرساختی است، ارزیابی آن صرفاً بر مبنای کیفیت پاسخ مدل‌ها انجام نمی‌شود؛ بلکه معیارهایی مانند درستی جریان‌های مالی، استحکام لایه‌ی دسترسی، پوشش آزمون، قابلیت توسعه و ویژگی‌های عملیاتی نیز اهمیت دارند.

در این فصل، ارزیابی در سه محور انجام می‌شود: ارزیابی فنی مبتنی بر آزمون‌ها، ارزیابی عملکرد معماری، و مقایسه‌ی دستاوردهای نهایی با اهداف اولیه.

۵-۲ روش ارزیابی

ارزیابی این پایان‌نامه بر سه منبع شواهد استوار است. منبع نخست، اجرای واقعی مجموعه‌آزمون خودکار برای سنجش درستی رفتار سرویس‌ها، نقاط پایانی و سناریوهای انتها به انتها است. منبع دوم، بازبینی قراردادهای قابل مشاهده‌ی سامانه است تا روشن شود دامنه‌ی API مستندشده با پیاده‌سازی واقعی سازگار است. منبع سوم نیز تحلیل مسیر بحرانی معماری است که نشان می‌دهد کدام بخش‌ها در مسیر هم‌زمان درخواست باقی مانده‌اند و کدام بخش‌ها به کش و صف منتقل شده‌اند.

این رویکرد آگاهانه انتخاب شده است؛ زیرا موضوع پایان‌نامه بیشتر درستی نرم‌افزاری و کفایت معماری را هدف می‌گیرد تا مقایسه‌ی سرعت مدل‌های هوش مصنوعی. در نتیجه، معیارهای این فصل به‌جای کیفیت پاسخ مدل‌ها، بر صحت قرارداد، پایداری مسیرهای مالی و قابلیت اتکای سامانه تمرکز دارند.

۵-۲-۱ محیط اجرا و داده‌های پایه

اجرای اصلی ارزیابی در تاریخ ۲۰۲۶-۰۳-۲۳ با فرمان php artisan test در محیط توسعه‌ی محلی پروژه انجام شده است. جدول ۵-۱ داده‌های پایه‌ی این اجرا را نشان می‌دهد.

جدول ۵-۱: مشخصات اجرای ارزیابی

شاخص	مقدار
نسخه‌ی بستر سمت سرور	Laravel 12 و PHP 8.2
فرمان ارزیابی اصلی	php artisan test
نتیجه‌ی کلی اجرا	موفق، بدون شکست در مجموعه‌آزمون
تعداد کل آزمون‌ها	۵۵ آزمون
تعداد کل Assertion‌ها	۱۶۰
زمان اجرای مجموعه‌آزمون	۹۱.۶۵ ثانیه
دامنه‌ی API	۳۵ مسیر شامل ۶ مسیر هوش مصنوعی و ۲۹ مسیر مدیریتی، مالی و
نسخه‌بندی شده	گزارشی

۵-۳ ارزیابی فنی

۵-۳-۱ آزمون‌های واحد

برای سنجش درستی اجزای مستقل سامانه، مجموعه‌ای از آزمون‌های واحد تعریف شده است. این آزمون‌ها روی سرویس‌ها، کارهای صف، سیاست‌های کش و اجزای ثبت گزارش متمرکز هستند. در کدپایه‌ی فعلی، ۱۳ فایل آزمون واحد وجود دارد که بخش‌های اصلی مختلفی را پوشش می‌دهند. برای مثال، WalletService و SubscriptionService برای اطمینان از درستی منطق مالی آزموده شده‌اند، InvoiceService ساخت فاکتور و اqlام آن را پوشش می‌دهد، UsageEstimationService

و UsageMeteringService به تخمین و محاسبه‌ی هزینه اختصاص دارند، GatewayLogService و مقصدهای گزارش خارجی پایداری لاگینگ را می‌سنجند و کارهای پس‌زمینه‌ای مانند تجمیع مصرف روزانه و پایش سلامت ارائه‌دهنده نیز به‌طور مستقل بررسی شده‌اند.

تمرکز این دسته از آزمون‌ها بر درستی منطق و رفتار اجزای منفرد است؛ بنابراین می‌توان خطاهای مربوط به محاسبه، تبدیل داده یا سیاست‌های کنترلی را پیش از درگیر شدن کل سامانه تشخیص داد.

۵-۳-۲ آزمون‌های ویژگی

در کنار آزمون‌های واحد، ۱۵ فایل آزمون ویژگی برای سنجش رفتار API و سناریوهای انتها به انتها تعریف شده است. این آزمون‌ها شامل گروه‌های مهمی مانند احراز هویت OTP، پروفایل و کش پروفایل، مدیریت مشتری و کلید API، کیف پول، طرح‌ها و اشتراک‌ها، فاکتورها و گزارش‌ها، مدیریت ارائه‌دهنده و مدل، نقاط پایانی اصلی درگاه هوش مصنوعی و همچنین لاگ ممیزی و سناریوهای کاربری ترکیبی را در بر می‌گیرند.

وجود این آزمون‌ها نشان می‌دهد که طراحی پروژه فقط در سطح کلاس‌ها متوقف نشده و رفتار واقعی سامانه در مسیرهای مهم نیز مورد بررسی قرار گرفته است.

۵-۳-۳ نتایج آزمون

به‌جای اکتفا به توصیف کیفی، نتیجه‌ی اجرای واقعی مجموعه آزمون در جدول ۵-۲ گزارش شده است. این داده‌ها نشان می‌دهند که پیاده‌سازی فعلی، دست‌کم در محدوده‌ی سناریوهای تعریف‌شده، از پایداری نرم‌افزاری مناسبی برخوردار است.

جدول ۵-۲: خلاصه‌ی اجرای مجموعه‌آزمون

شاخص	مقدار	توضیح
فایل‌های آزمون واحد	۱۳	پوشش سرویس‌های مالی، تخمین مصرف، کش، لاگینگ و کارهای پس‌زمینه
فایل‌های آزمون ویژگی	۱۵	پوشش احراز هویت، مدیریت کلید، کیف پول، اشتراک، گزارش و مسیرهای درگاه
تعداد کل آزمون‌ها	۵۵	شامل آزمون‌های واحد و ویژگی
تعداد کل Assertion‌ها	۱۶۰	نشان‌دهنده‌ی آن است که هر آزمون به یک بررسی سطحی محدود نشده است
وضعیت نهایی	موفق	همه‌ی آزمون‌ها در اجرای داوری با موفقیت عبور کرده‌اند

۵-۳-۴ سناریوهای کلیدی تأییدشده

برای ارزیابی یک سامانه‌ی واسط، صرف تعداد آزمون‌ها کافی نیست؛ مهم‌تر آن است که آیا مسیرهای بحرانی واقعاً سنجیده شده‌اند یا خیر. جدول ۵-۳ تعدادی از سناریوهای مهمی را که در کدپایه به‌صورت خودکار بررسی شده‌اند نشان می‌دهد.

جدول ۵-۳: نمونه‌ای از سناریوهای کلیدی تأییدشده در آزمون‌ها

سناریو	شاهد آزمون	نتیجه
شروع و تأیید احراز هویت OTP	AuthOtpTest	جریان تولید چالش، تأیید کد و صدور توکن نشست بررسی شده است.
رد درخواست فاقد کلید یا دارای داده‌ی نامعتبر	GatewayTest	درگاه برای نبود کلید، نبود ارائه‌دهنده و خطای اعتبارسنجی پاسخ استاندارد تولید می‌کند.
جلوگیری از مصرف بدون موجودی کافی	UserScenarioTest	در صورت کمبود موجودی، درخواست پیش از فراخوانی ارائه‌دهنده متوقف می‌شود.
پشتیبانی از مصرف مبتنی بر اشتراک	UserScenarioTest	کاربر دارای اشتراک معتبر می‌تواند بدون برداشت مستقیم از کیف پول از سرویس استفاده کند.
نرمال‌سازی تفاوت ارائه‌دهندگان	GatewayTest	پاسخ‌های Gemini به قالب سازگار با OpenAI برگردانده می‌شوند و مسیر Groq نیز بررسی شده است.
تولید خروجی‌های جانبی و قابلیت استفاده‌ی ابزاری	InvoiceTest و UserScenarioTest	تولید PDF فاکتور و در دسترس بودن Playground تأیید شده‌اند.

۴-۵ ارزیابی عملکرد

۱-۴-۵ تأخیر پاسخ

در این پایان‌نامه پنج‌مارک میدانی مستقل برای اندازه‌گیری تأخیر و توان عملیاتی انجام نشده است؛ بنابراین ارائه‌ی اعداد مطلق برای latency یا throughput از نظر روش‌شناختی موجه نیست. با این حال، تحلیل مسیر بحرانی سامانه نشان می‌دهد که طراحی انجام‌شده تلاش کرده است سربار درون‌سامانه‌ای را

در بخش‌های قابل کنترل کاهش دهد. برای مثال، داده‌های پرتکرار مانند ارائه‌دهندگان، مدل‌ها، طرح‌ها و پروفایل کاربر از کش خوانده می‌شوند، کارهای غیر بحرانی مانند ارسال گزارش خارجی، تولید PDF و ارسال OTP از مسیر اصلی درخواست خارج و به صف منتقل شده‌اند، و استفاده از الگوی خط لوله سربار منطقی سامانه را به مراحل کوچک و ساده تقسیم کرده و پیچیدگی هر مرحله را محدود نگه داشته است. بر این اساس، می‌توان گفت که معماری سامانه برای کاهش سربار مسیر اصلی **طراحی شده است**، اما ادعای عددی درباره‌ی کارایی نهایی آن نیازمند آزمون بار مستقل در محیطی نزدیک به بهره‌برداری است. این تمایز برای پرهیز از بیش‌اظهاری در ارزیابی ضروری است.

۵-۴-۲ دقت صورت حساب گذاری

دقت مالی در این پروژه از دو مسیر تقویت شده است. سامانه پیش از ارسال درخواست، با استفاده از توکن ساز BPE و معیارهای غیرتوکنی، هزینه‌ی احتمالی را محاسبه می‌کند تا بتواند تصمیم مالی اولیه را بگیرد. پس از دریافت پاسخ نیز، اگر داده‌ی مصرف از سوی ارائه‌دهنده موجود باشد همان مقدار واقعی ملاک قرار می‌گیرد و در غیر این صورت، تخمین اولیه به عنوان پشتیبان باقی می‌ماند.

این طراحی دو مزیت مهم دارد. نخست، از ارسال درخواست‌هایی که موجودی کافی ندارند جلوگیری می‌شود. دوم، حتی در صورت نبود فراساختار مصرف در پاسخ ارائه‌دهنده، سامانه همچنان قادر است مدل مالی یکنواختی ارائه دهد. از منظر شواهد نرم‌افزاری، آزمون‌های UsageEstimationServiceTest، WalletServiceTest، SubscriptionServiceTest و UserScenarioTest این منطق را در سطح سرویس و سناریوی کاربری بررسی می‌کنند. با این حال، تطبیق نهایی با صورت حساب واقعی همه‌ی ارائه‌دهندگان هنوز نیازمند اعتبارسنجی میدانی و داده‌های عملیاتی بلندمدت است.

۵-۵ مقایسه با اهداف

در فصل اول، شش هدف عملیاتی برای پروژه تعریف شد. جدول ۵-۴ میزان تحقق این اهداف را نشان می‌دهد.

جدول ۵-۴: مقایسه‌ی اهداف اولیه با وضعیت نهایی پیاده‌سازی

هدف	وضعیت	شاهد و توضیح
ارائه‌ی رابط یکپارچه‌ی سازگار با OpenAI	کامل	۶ مسیر اصلی هوش مصنوعی در API موجود است و آزمون‌های GatewayTest رفتار آن‌ها را بررسی می‌کنند.
پشتیبانی از چند ارائه‌دهنده	کامل	چهار ارائه‌دهنده پشتیبانی می‌شوند؛ با این توضیح که پوشش قابلیت‌های Gemini در نسخه‌ی فعلی به گفتگو، responses و جاسازی متن محدود است.
احراز هویت و مدیریت دسترسی	کامل	مسیرهای OTP، مشتریان API، کلیدها و لاگ ممیزی وجود دارند و در آزمون‌های ویژگی پوشش داده شده‌اند.
سیستم صورت‌حساب و کیف پول	کامل	سرویس‌های کیف پول، اشتراک و فاکتور به‌همراه سناریوهای بدهکارسازی و کنترل موجودی آزموده شده‌اند.
گزارش‌دهی و مانیتورینگ	کامل	گزارش مصرف، دفترکل کیف پول، گزارش فاکتورها و دو مقصد گزارش خارجی در کدپایه و آزمون‌ها حاضر هستند.
زیرساخت عملیاتی کامل بیرونی	نسبی	منطق ارسال OTP و تولید PDF وجود دارد، اما اتصال به سرویس واقعی پیامک/ایمیل و درگاه پرداخت بیرونی هنوز تکمیل نشده است.

بر پایه‌ی این مقایسه می‌توان گفت که اهداف اصلی پروژه محقق شده‌اند و نقاط باقی‌مانده بیشتر به یکپارچه‌سازی با سرویس‌های عملیاتی بیرونی مربوط می‌شوند، نه به منطق مرکزی سامانه. در عین حال، اعتبار این نتیجه‌گیری بیشتر در حوزه‌ی **درستی رفتاری و انسجام معماری** است و نه در حوزه‌ی **کارایی میدانی در مقیاس تولید**.

فصل ششم

نتیجه گیری

۱-۶ جمع بندی

این پایان نامه به مسئله‌ی یکپارچه سازی سرویس های هوش مصنوعی از منظر فنی، مالی و مدیریتی پرداخت. مسئله‌ی اصلی آن بود که سازمان ها و توسعه دهندگان، برای استفاده از چند ارائه دهنده‌ی هوش مصنوعی، ناچار به پذیرش چندگانگی در API، مدل مالی، گزارش گیری و کنترل دسترسی هستند. برای پاسخ به این مسئله، یک سامانه‌ی واسط مبتنی بر Laravel طراحی و پیاده سازی شد که رابطی سازگار با OpenAI را در سطح بیرونی ارائه می دهد و در سطح درونی، قابلیت های احراز هویت، مدیریت کلید، کیف پول، اشتراک، فاکتور، گزارش گیری و ارسال گزارش خارجی را یکپارچه می کند.

معماری مبتنی بر خط لوله، لایه بندی شفاف سرویس ها و استفاده از کش و صف، سبب شده است که سامانه نه تنها از منظر توسعه پذیری و آزمون پذیری مناسب باشد، بلکه از دید بهره برداری نیز قابلیت رشد داشته باشد. همچنین، پشتیبانی از ارائه دهنده های مانند Gemini که از قرارداد بومی متفاوتی استفاده می کند، نشان داد که معماری انتخاب شده برای تطبیق با ناهمگنی ارائه دهندگان کفایت دارد. در ارزیابی انجام شده، ۵۵ آزمون خودکار با ۱۶۰ Assertion با موفقیت اجرا شدند؛ بنابراین می توان گفت که نسخه‌ی فعلی از نظر درستی مسیرهای اصلی، شواهد مناسبی ارائه می دهد. با این حال، این نتیجه گیری به معنای اثبات کارایی میدانی در مقیاس تولید نیست.

۲-۶ دستاوردها

مهم ترین دستاورد این پروژه، طراحی و پیاده سازی یک درگاه سازگار با OpenAI برای دسترسی به چند ارائه دهنده‌ی هوش مصنوعی در قالب شش مسیر اصلی هوش مصنوعی است. این دستاورد تنها به لایه‌ی درگاهی محدود نمی شود؛ زیرا در کنار آن سازوکاری یکپارچه برای مدیریت کاربران، مشتریان و کلیدهای

API نیز ایجاد شده است.

از منظر مالی و عملیاتی نیز پروژه به یک مدل ترکیبی شامل کیف پول، اشتراک، فاکتور و گزارش مصرف رسیده و منطق تخمین و اندازه‌گیری مصرف را هم برای توکن و هم برای معیارهای غیرتوکنی توسعه داده است. افزوده شدن گزارش‌گیری داخلی و ارسال لاگ ساخت‌یافته به سرویس‌های خارجی، در کنار ۲۸ فایل آزمون و ۵۵ آزمون خودکار، باعث شده سامانه فقط یک نمونه‌ی مفهومی نباشد. وجود Playground و مستندات API نیز استفاده و توسعه‌ی بعدی آن را آسان‌تر کرده است.

۳-۶ محدودیت‌ها

با وجود تحقق اهداف اصلی، نسخه‌ی فعلی سامانه همچنان دارای محدودیت‌هایی است: ارسال OTP در سطح منطق کامل شده است، اما هنوز به سرویس واقعی پیامک یا ایمیل متصل نیست. به همین ترتیب، شارژ کیف پول نیز به درگاه پرداخت واقعی متصل نشده و در نسخه‌ی فعلی به‌صورت دستی انجام می‌شود. قواعد راهبری پیشرفته در جدول routing_rules پیش‌بینی شده‌اند، اما انتخاب ارائه‌دهنده هنوز به‌صورت صریح توسط مشتری انجام می‌شود و سامانه تصمیم‌گیری هوشمند را بر عهده نگرفته است.

از سوی دیگر، پشتیبانی از Gemini در همه‌ی نقاط پایانی چندرسانه‌ای تعمیم نیافته است و ارزیابی عملکرد این پایان‌نامه نیز بیشتر بر تحلیل معماری و آزمون‌های نرم‌افزاری تکیه دارد. بنابراین بنچ‌مارک میدانی گسترده، آزمون بار و تطبیق بلندمدت با صورت‌حساب واقعی ارائه‌دهندگان هنوز خارج از دامنه‌ی نسخه‌ی فعلی قرار می‌گیرند.

۴-۶ کارهای آتی

برای توسعه‌ی بعدی سامانه، چند مسیر روشن وجود دارد. مهم‌ترین آن‌ها اتصال OTP به سرویس‌های واقعی پیامک یا ایمیل و یکپارچه‌سازی با درگاه پرداخت برای شارژ خودکار کیف پول و تسویه‌ی اشتراک است. در سطح معماری نیز پیاده‌سازی راهبردهای هوشمند انتخاب ارائه‌دهنده، مانند انتخاب ارزان‌ترین، سریع‌ترین یا کم‌بارترین مسیر، می‌تواند گام بعدی طبیعی باشد.

افزودن قابلیت streaming برای پاسخ‌های برخط، توسعه‌ی پشتیبانی چندرسانه‌ای کامل‌تر برای همه‌ی ارائه‌دهندگان، اضافه کردن معیارهای مشاهده‌پذیری پیشرفته‌تر مانند ردگیری توزیع‌شده و

داشبوردهای عملیاتی، و در نهایت انجام آزمون‌های بار و تحلیل تجربی عملکرد در سناریوهای واقعی سازمانی، مسیرهایی هستند که می‌توانند نسخه‌ی فعلی را به یک سامانه‌ی آماده‌تر برای بهره‌برداری گسترده تبدیل کنند.

در مجموع، پروژه‌ی حاضر نشان می‌دهد که طراحی یک سامانه‌ی واسط برای تجمیع سرویس‌های هوش مصنوعی، نه فقط از منظر فنی شدنی است، بلکه می‌تواند به عنوان مبنایی قابل توسعه برای مدیریت مصرف هوش مصنوعی در سطح سازمان مورد استفاده قرار گیرد؛ مشروط بر آن که در گام بعدی، یکپارچه‌سازی‌های بیرونی و ارزیابی میدانی آن تکمیل شود.

کتاب نامه

- [1] Anyscale. Anyscale Endpoints documentation. <https://docs.anyscale.com/endpoints/>, 2024. Accessed: 2025-01-29.
- [2] Better Stack. Better Stack Logs documentation. <https://betterstack.com/docs/logs/>, 2024. Accessed: 2025-01-29.
- [3] DarkaOnLine. L5-Swagger: Swagger integration for Laravel. <https://github.com/DarkaOnLine/L5-Swagger>, 2024. Accessed: 2025-01-29.
- [4] Evan You and the Vue.js Team. Vue.js — the progressive JavaScript framework. <https://vuejs.org>, 2024. Accessed: 2025-01-29.
- [5] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Phd dissertation, University of California, Irvine, 2000.
- [6] Google DeepMind. Gemini API documentation. <https://ai.google.dev/docs>, 2024. Accessed: 2025-01-29.
- [7] Grafana Labs. Grafana Loki — like prometheus, but for logs. <https://grafana.com/oss/loki/>, 2024. Accessed: 2025-01-29.
- [8] Groq. GroqCloud API documentation. <https://console.groq.com/docs>, 2024. Accessed: 2025-01-29.
- [9] Laravel LLC. Laravel Sanctum documentation. <https://laravel.com/docs/sanctum>, 2024. Accessed: 2025-01-29.

- [10] Laravel LLC. Laravel — the PHP framework for web artisans. <https://laravel.com/docs>, 2024. Accessed: 2025-01-29.
- [11] Microsoft Corporation. SQL Server documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>, 2024. Accessed: 2025-01-29.
- [12] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2015.
- [13] OpenAI. OpenAI platform documentation. <https://platform.openai.com/docs>, 2024. Accessed: 2025-01-29.
- [14] OpenRouter. Unified AI API platform. <https://openrouter.ai/docs>, 2024. Accessed: 2025-01-29.
- [15] Redis Ltd. Redis documentation. <https://redis.io/docs>, 2024. Accessed: 2025-01-29.
- [16] Chris Richardson. *Microservices Patterns*. Manning Publications, 2018.
- [17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

پیوست آ

مستندات API

در این پیوست، قرارداد بیرونی سامانه به صورت منظم مستند شده است. فهرست مسیرها مستقیماً از فایل مسیرها، کنترلرها، منابع‌های پاسخ و آزمون‌های خودکار پروژه استخراج شده است؛ بنابراین این پیوست نه یک توصیف آرمانی، بلکه شرحی از رفتار واقعی نسخه‌ی پیاده‌سازی شده است.

آ-۱ قراردادهای مشترک

احراز هویت و سربرگ‌ها

مسیرهای کاربری و مدیریتی مانند `/me`، `/wallet`، `/plans` و `/reports` از توکن Sanctum کاربر استفاده می‌کنند و قالب رایج سربرگ آن‌ها `Authorization: Bearer <SANCTUM\_TOKEN>` است. مسیرهای هوش مصنوعی از کلید پروژه استفاده می‌کنند و دو قالب `Authorization: Bearer <API\_KEY>` و `X-API-Key: <API\_KEY>` را می‌پذیرند. همچنین مسیر `GET /api/v1/invoices/{invoice}/pdf` بر پایه‌ی پیوند امضاشده عمل می‌کند و معمولاً از فیلد `pdf_url` در پاسخ فاکتور به‌دست می‌آید.

نحوه‌ی بازگشت Resource Collection

در این پروژه `JsonResource::withoutWrapping()` فعال شده است؛ بنابراین پاسخ‌های مجموعه‌ای با لفافه‌ی `data` برگردانده نمی‌شوند و مستقیماً به صورت آرایه‌ی JSON در خروجی قرار می‌گیرند.

الگوی خطا در مسیرهای عمومی

مسیرهای عمومی و مدیریتی خطاها را معمولاً با یک فیلد ساده‌ی `message` بازمی‌گردانند.

```

1 {
2   "message": "The name field is required."
3 }

```

کد آ-۱: نمونه خطا در مسیرهای عمومی

الگوی خطا در مسیرهای هوش مصنوعی

درگاه هوش مصنوعی برای سازگاری بیرونی، خطاها را در قالبی مشابه OpenAI بازمی‌گرداند.

```

1 {
2   "error": {
3     "message": "Provider and model are required.",
4     "type": "invalid_request_error",
5     "param": "provider",
6     "code": null
7   }
8 }

```

کد آ-۲: نمونه خطا در مسیرهای هوش مصنوعی

آ-۲ نقاط پایانی احراز هویت

آ-۲-۱ POST /api/v1/auth/otp/start

کارکرد. آغاز فرایند احراز هویت با ارسال رمز یک‌بارمصرف.

احراز هویت. نیاز ندارد.

ساختار درخواست. بدنه‌ی درخواست شامل دو فیلد اجباری channel و destination است. مقدار

channel فقط می‌تواند email یا sms باشد.

ساختار پاسخ موفق. یک شیء ساده با فیلد status که در حالت موفق مقدار آن sent است.

```

1 POST /api/v1/auth/otp/start
2 Content-Type: application/json
3
4 {
5   "channel": "email",
6   "destination": "user@example.com"
7 }

```

کد آ-۳: نمونه درخواست OTP Start

```

1 {
2   "status": "sent"
3 }

```

کد آ-۴: نمونه پاسخ OTP Start

آ-۲-۲ POST /api/v1/auth/otp/verify

کارکرد: تأیید رمز یک بار مصرف و صدور توکن دسترسی داشبورد.

احراز هویت. نیاز ندارد.

ساختار درخواست. فیلدهای اجباری destination و code و فیلد اختیاری channel دریافت می‌شوند.

ساختار پاسخ موفق. شیئی شامل token, token_type و user. شیء user دارای فیلدهای id, name, email, phone, status, profile_image_url, created_at و updated_at است.

```

1 POST /api/v1/auth/otp/verify
2 Content-Type: application/json
3
4 {
5   "destination": "user@example.com",

```



```
6  "code": "123456",
7  "channel": "email"
8 }
```

کد آ-۵: نمونه درخواست OTP Verify

```
1 {
2   "token": "1|wQyOfP4...sanctum-token",
3   "token_type": "Bearer",
4   "user": {
5     "id": 10,
6     "name": "User Example",
7     "email": "user@example.com",
8     "phone": null,
9     "status": "active",
10    "profile_image_url": null,
11    "created_at": "2026-01-29T08:00:00.000000Z",
12    "updated_at": "2026-01-29T08:00:00.000000Z"
13  }
14 }
```

کد آ-۶: نمونه پاسخ OTP Verify

آ-۳ نقاط پایانی کاربر

آ-۳-۱ GET /api/v1/me

کارکرد: دریافت پروفایل کاربر جاری.

احراز هویت. توکن Sanctum.

ساختار درخواست. این مسیر بدنه ندارد.

ساختار پاسخ موفق. پاسخ این مسیر یک شیء کاربر است. فیلدهای اصلی آن id, name, email, phone, status, profile_image_url, created_at و updated_at هستند.

آ-۳-۲ PATCH /api/v1/me

کارکرد. بهروزرسانی پروفایل کاربر.

احراز هویت. توکن Sanctum.

ساختار درخواست. فیلدهای name, email و phone اختیاری هستند. در صورت ارسال تصویر، فیلد profile_image_url باید به صورت فایل تصویری و در قالب multipart/form-data ارسال شود. ساختار پاسخ موفق. همان ساختار مسیر GET /api/v1/me با مقادیر به روز شده.

آ-۴-۱ نقاط پایانی کلیدهای API

آ-۴-۱ GET /api/v1/api-clients

کارکرد. دریافت فهرست مشتریان API متعلق به کاربر جاری.

احراز هویت. توکن Sanctum.

ساختار درخواست. این مسیر بدنه ندارد.

ساختار پاسخ موفق. آرایه‌ای از اشیاء با فیلدهای id, name, status, created_at و updated_at.

آ-۴-۲ POST /api/v1/api-clients

کارکرد. ایجاد مشتری API جدید.

احراز هویت. توکن Sanctum.

ساختار درخواست. تنها فیلد اجباری این مسیر name است.

ساختار پاسخ موفق. یک شیء از نوع API Client با فیلدهای id, name, status, created_at و

updated_at.

```
1 POST /api/v1/api-clients
2 Authorization: Bearer <SANCTUM_TOKEN>
3
```

```

4 {
5   "name": "Primary Client"
6 }

```

کد آ-۷: نمونه درخواست Create API Client

```

1 {
2   "id": 1,
3   "name": "Primary Client",
4   "status": "active",
5   "created_at": "2026-01-29T08:00:00.000000Z",
6   "updated_at": "2026-01-29T08:00:00.000000Z"
7 }

```

کد آ-۸: نمونه پاسخ Create API Client

آ-۴-۳ GET /api/v1/api-clients/{apiClient}/keys

کارکرد: دریافت فهرست کلیدهای ثبت شده برای یک مشتری API.

احراز هویت: توکن Sanctum.

ساختار درخواست: شناسه‌ی مشتری در مسیر قرار می‌گیرد و بدنه‌ای ارسال نمی‌شود.

ساختار پاسخ موفق: پاسخ این مسیر آرایه‌ای از فراداده‌ی کلیدها است. هر عضو آرایه معمولاً

فیلدهای id, key_prefix, scopes, rate_limit_per_min, allowed_ips, expires_at, revoked_at و

created_at را دارد. مقدار واقعی کلید در این مسیر هرگز برگردانده نمی‌شود.

آ-۴-۴ POST /api/v1/api-clients/{apiClient}/keys

کارکرد: ایجاد کلید API جدید برای یک مشتری.

احراز هویت: توکن Sanctum.

ساختار درخواست. در این مسیر می‌توان آرایه‌ی اختیاری scopes را برای حوزه‌های دسترسی، فیلد rate_limit_per_min را برای تعیین سقف درخواست در دقیقه، allowed_ips را برای فهرست آدرس‌های مجاز و expires_at را برای تاریخ انقضای کلید ارسال کرد.

ساختار پاسخ موفق. شیئی شامل api_key، key_prefix، expires_at و resource. فیلد api_key فقط یک‌بار بازگردانده می‌شود.

```

1 POST /api/v1/api-clients/1/keys
2 Authorization: Bearer <SANCTUM_TOKEN>
3
4 {
5   "scopes": ["ai:chat", "ai:embeddings"],
6   "rate_limit_per_min": 60,
7   "allowed_ips": ["127.0.0.1"]
8 }
```

کد آ-۹: نمونه درخواست Create API Key

```

1 {
2   "api_key": "gk_abcd1234secretvalue",
3   "key_prefix": "abcd1234",
4   "expires_at": null,
5   "resource": {
6     "id": 7,
7     "key_prefix": "abcd1234",
8     "scopes": ["ai:chat", "ai:embeddings"],
9     "rate_limit_per_min": 60,
10    "allowed_ips": ["127.0.0.1"],
11    "expires_at": null,
12    "revoked_at": null,
13    "created_at": "2026-01-29T08:00:00.000000Z"
14  }
15 }
```

کد آ-۱۰: نمونه پاسخ Create API Key

آ-۴-۵ POST /api/v1/api-keys/{apiKey}/revoke

کارکرد: لغو کلید API.

احراز هویت: توکن Sanctum.

ساختار درخواست: این مسیر بدنه ندارد و شناسه‌ی کلید در مسیر ارسال می‌شود.

ساختار پاسخ موفق: شیئی با فیلد status و مقدار revoked.

آ-۴-۶ POST /api/v1/api-keys/{apiKey}/rotate

کارکرد: چرخش کلید API و صدور کلید جدید با ابطال کلید پیشین.

احراز هویت: توکن Sanctum.

ساختار درخواست: این مسیر بدنه ندارد.

ساختار پاسخ موفق: همانند مسیر ایجاد کلید، شامل api_key، expires_at، prefix و key.

resource.

آ-۵-۵ نقاط پایانی هوش مصنوعی

آ-۵-۱ POST /api/v1/ai/responses

کارکرد: ارسال درخواست در قالب Responses API سازگار با OpenAI.

احراز هویت: کلید پروژه از طریق Authorization یا X-API-Key.

ساختار درخواست: فیلدهای اجباری provider و model و حداقل یک ورودی در input. سایر

فیلدهای سازگار با Responses API به سمت ارائه‌دهنده عبور داده می‌شوند.

ساختار پاسخ موفق: شیئی از نوع response شامل فیلدهایی مانند id، object، model و output.

در صورت وجود اطلاعات مصرف، فیلد usage نیز بازگردانده می‌شود.

```
1 POST /api/v1/ai/responses
2 Authorization: Bearer <API_KEY>
3
```

```
4 {  
5   "provider": "openai",  
6   "model": "gpt-4o-mini",  
7   "input": "Hello"  
8 }
```

کد آ-۱۱: نمونه درخواست Responses API

```
1 {  
2   "id": "resp_test",  
3   "object": "response",  
4   "model": "gpt-4o-mini",  
5   "output": [  
6     {  
7       "type": "message",  
8       "role": "assistant",  
9       "content": [  
10        {  
11          "type": "output_text",  
12          "text": "Hello from responses"  
13        }  
14      ]  
15    }  
16  ]  
17 }
```

کد آ-۱۲: نمونه پاسخ Responses API

POST /api/v1/ai/chat/completions آ-۵-۲

کارکرد: ارسال درخواست تکمیل گفتگو.

احراز هویت. کلید پروژه.

ساختار درخواست. فیلدهای اجباری provider, model و messages. آرایه‌ی messages باید حداقل یک پیام در قالب content/role داشته باشد.

ساختار پاسخ موفق. شیئی از نوع chat.completion شامل id, object, created, model, choices و usage. این پاسخ هم برای OpenAI-مانندها و هم برای Gemini پس از نرمال‌سازی به همین شکل برمی‌گردد.

```
1 POST /api/v1/ai/chat/completions
2 Authorization: Bearer <API_KEY>
3
4 {
5   "provider": "groq",
6   "model": "llama-3.3-70b-versatile",
7   "messages": [
8     {"role": "user", "content": "Hello!"}
9   ]
10 }
```

کد آ-۱۳: نمونه درخواست Chat Completions

```
1 {
2   "id": "chatcmpl_groq",
3   "object": "chat.completion",
4   "created": 1738137600,
5   "model": "llama-3.3-70b-versatile",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "Hello from Groq"
12      },
```

```

13     "finish_reason": "stop"
14   }
15 ],
16 "usage": {
17   "prompt_tokens": 5,
18   "completion_tokens": 7,
19   "total_tokens": 12
20 }
21 }

```

کد آ-۱۴: نمونه پاسخ Chat Completions

آ-۵-۳ POST /api/v1/ai/embeddings

کارکرد: ارسال درخواست جاسازی متن.

احراز هویت: کلید پروژه.

ساختار درخواست: فیلدهای اجباری provider, model, input و مقدار input می‌تواند رشته یا

آرایه‌ای از رشته‌ها باشد.

ساختار پاسخ موفق: شیئی با object=list، آرایه‌ی data، فیلد model و در صورت وجود، usage.

```

1 POST /api/v1/ai/embeddings
2 X-API-Key: <API_KEY>
3
4 {
5   "provider": "openai",
6   "model": "text-embedding-3-small",
7   "input": "hello"
8 }

```

کد آ-۱۵: نمونه درخواست Embeddings


```

1 {
2   "object": "list",
3   "data": [
4     {
5       "object": "embedding",
6       "index": 0,
7       "embedding": [0.1, 0.2]
8     }
9   ],
10  "model": "text-embedding-3-small",
11  "usage": {
12    "prompt_tokens": 4,
13    "total_tokens": 4
14  }
15 }

```

کد آ-۱۶: نمونه پاسخ Embeddings

آ-۵-۴ POST /api/v1/ai/images/generations

کارکرد: ارسال درخواست تولید تصویر.

احراز هویت: کلید پروژه.

ساختار درخواست: فیلدهای اجباری provider، model و prompt. فیلدهای سازگار دیگری مانند

n یا size نیز می‌توانند همراه درخواست عبور داده شوند.

ساختار پاسخ موفق: شیئی شامل زمان ایجاد در created و آرایه‌ی data که هر عضو آن معمولاً

حاوی url یا داده‌ی تصویر است.

```

1 POST /api/v1/ai/images/generations
2 Authorization: Bearer <API_KEY>
3
4 {

```

```

5  "provider": "openai",
6  "model": "gpt-image-1",
7  "prompt": "A test image"
8  }

```

کد آ-۱۷: نمونه درخواست Image Generation

```

1  {
2    "created": 1738137600,
3    "data": [
4      {
5        "url": "https://example.com/image.png"
6      }
7    ]
8  }

```

کد آ-۱۸: نمونه پاسخ Image Generation

آ-۵-۵ POST /api/v1/ai/audio/transcriptions

کارکرد. ارسال درخواست تبدیل گفتار به متن.

احراز هویت. کلید پروژه.

ساختار درخواست. فیلدهای اجباری provider و model به همراه فایل بارگذاری شده در file. این مسیر باید به صورت multipart/form-data فراخوانی شود. بر اساس منطق درگاه، نبود فیلد file مستقیماً به خطای invalid_request_error منجر می شود.

ساختار پاسخ موفق. در رایج ترین حالت شیئی شامل فیلد text که متن رونویسی شده را نگه می دارد.

```

1  {
2    "text": "hello world"
3  }

```

کد آ-۱۹: نمونه پاسخ Audio Transcription

آ-۵-۶ POST /api/v1/ai/audio/speech

کارکرد: ارسال درخواست تبدیل متن به گفتار.

احراز هویت: کلید پروژه.

ساختار درخواست: فیلدهای اجباری provider, model, input و voice.

ساختار پاسخ موفق: این مسیر معمولاً JSON برنمی‌گرداند، بلکه پاسخ دودویی صوت به همراه

Content-Type مناسب، مانند audio/mpeg، تحویل می‌دهد.

```
1 POST /api/v1/ai/audio/speech
2 Authorization: Bearer <API_KEY>
3
4 {
5   "provider": "openai",
6   "model": "gpt-4o-mini-tts",
7   "input": "Hello there",
8   "voice": "alloy"
9 }
```

کد آ-۲۰: نمونه درخواست Audio Speech

```
1 HTTP/1.1 200 OK
2 Content-Type: audio/mpeg
3
4 <binary audio bytes>
```

کد آ-۲۱: نمونه پاسخ Audio Speech

آ-۶ نقاط پایانی مالی

آ-۶-۱ GET /api/v1/wallet

کارکرد: مشاهده‌ی موجودی کیف پول.

احراز هویت: توکن Sanctum.

ساختار درخواست: این مسیر بدنه ندارد.

ساختار پاسخ موفق: شیئی با فیلدهای id, balance, currency و updated_at.

آ-۶-۲ POST /api/v1/wallet/topup

کارکرد: افزایش موجودی کیف پول در نسخه‌ی فعلی از مسیر داخلی سامانه.

احراز هویت: توکن Sanctum.

ساختار درخواست: فیلد اجباری amount و فیلد اختیاری reason.

ساختار پاسخ موفق: شیئی شامل دو بخش wallet و transaction. بخش wallet موجودی جدید

را نشان می‌دهد و بخش transaction شامل id, type, amount, reason, ref_type, ref_id, meta و created_at است.

```
1 POST /api/v1/wallet/topup
2 Authorization: Bearer <SANCTUM_TOKEN>
3
4 {
5     "amount": 25.50,
6     "reason": "test_topup"
7 }
```

کد آ-۲۲: نمونه درخواست Wallet Topup

```
1 {
2     "wallet": {
3         "id": 1,
4         "balance": 25.5,
```

```

5     "currency": "USD",
6     "updated_at": "2026-01-29T08:00:00.000000Z"
7 },
8 "transaction": {
9     "id": 1,
10    "type": "credit",
11    "amount": 25.5,
12    "reason": "test_topup",
13    "ref_type": null,
14    "ref_id": null,
15    "meta": null,
16    "created_at": "2026-01-29T08:00:00.000000Z"
17 }
18 }

```

کد آ-۲۳: نمونه پاسخ Wallet Topup

آ-۶-۳ GET /api/v1/wallet/transactions

کارکرد: دریافت دفترکل تراکنش‌های کیف پول.

احراز هویت: توکن Sanctum.

ساختار درخواست: پارامتر اختیاری limit در query string پذیرفته می‌شود.

ساختار پاسخ موفق: آرایه‌ای از اشیای تراکنش با همان ساختار transaction در مسیر افزایش

موجودی.

آ-۶-۴ GET /api/v1/plans

کارکرد: مشاهده لیست طرح‌های اشتراک.

احراز هویت: توکن Sanctum.

ساختار درخواست: این مسیر بدنه ندارد.

ساختار پاسخ موفق. آرایه‌ای از اشیاء با فیلدهای `id`, `name`, `price`, `currency`, `period`, `created_at` و `updated_at`, `status`, `features`, `rate_limits`, `included_credits`.

آ-۶-۵ POST /api/v1/plans

کارکرد. ایجاد طرح اشتراک جدید.

احراز هویت. توکن Sanctum.

ساختار درخواست. فیلدهای `name`, `price` و `period` اجباری هستند. در کنار آن‌ها، فیلدهای `currency`, `included_credits`, `rate_limits`, `features` و `status` نیز به صورت اختیاری پذیرفته می‌شوند. ساختار پاسخ موفق. شیء طرح اشتراک با همان فیلدهای مسیر `GET /api/v1/plans`.

آ-۶-۶ POST /api/v1/subscriptions

کارکرد. خرید اشتراک.

احراز هویت. توکن Sanctum.

ساختار درخواست. تنها فیلد اجباری `plan_id` است.

ساختار پاسخ موفق. شیئی شامل `id`, `status`, `starts_at`, `ends_at`, `renewal_at` و `canceled_at` و شیء تو در توی `plan`.

```
1 POST /api/v1/subscriptions
2 Authorization: Bearer <SANCTUM_TOKEN>
3
4 {
5   "plan_id": 1
6 }
```

کد آ-۶-۲۴: نمونه درخواست Create Subscription

```
1 {
2   "id": 25,
3   "status": "active",
4   "starts_at": "2026-01-29T08:00:00.000000Z",
```

```
5 "ends_at": "2026-02-29T08:00:00.000000Z",
6 "renewal_at": "2026-02-29T08:00:00.000000Z",
7 "canceled_at": null,
8 "plan": {
9   "id": 1,
10  "name": "Starter",
11  "price": 10,
12  "currency": "USD",
13  "period": "monthly",
14  "included_credits": 100,
15  "rate_limits": null,
16  "features": null,
17  "status": "active",
18  "created_at": "2026-01-29T08:00:00.000000Z",
19  "updated_at": "2026-01-29T08:00:00.000000Z"
20 }
21 }
```

کد آ-۲۵: نمونه پاسخ Create Subscription

آ-۶-۷ GET /api/v1/subscriptions/current

کارکرد. دریافت اشتراک فعال کاربر.

احراز هویت. توکن Sanctum.

ساختار درخواست. این مسیر بدنه ندارد.

ساختار پاسخ موفق. همان ساختار مسیر POST /api/v1/subscriptions در صورت نبود اشتراک

فعال، خطای 404 با پیام No active subscription. برگردانده می‌شود.

آ-۶-۸ POST /api/v1/subscriptions/cancel

کارکرد. لغو اشتراک فعال.

احراز هویت. توکن Sanctum.
 ساختار درخواست. این مسیر بدنه ندارد.
 ساختار پاسخ موفق. همان ساختار منبع اشتراک با status برابر canceled. در صورت نبود اشتراک فعال، خطای 404 بازگردانده می‌شود.

GET /api/v1/invoices ۹-۶-آ

کارکرد. مشاهده‌ی لیست فاکتورها.
 احراز هویت. توکن Sanctum.
 ساختار درخواست. پارامتر اختیاری limit پذیرفته می‌شود.
 ساختار پاسخ موفق. آرایه‌ای از خلاصه‌ی فاکتورها با فیلدهای id, number, status, currency, subtotal, tax, total, issued_at, paid_at, pdf_url, created_at و updated_at. در مسیر فهرست، اقلام ریزفاکتور معمولاً بارگذاری نمی‌شوند.

GET /api/v1/invoices/{invoice} ۱۰-۶-آ

کارکرد. مشاهده‌ی جزئیات یک فاکتور مشخص.
 احراز هویت. توکن Sanctum.
 ساختار درخواست. شناسه‌ی فاکتور در مسیر قرار می‌گیرد.
 ساختار پاسخ موفق. شیء فاکتور با همان فیلدهای مسیر فهرست، به‌علاوه‌ی آرایه‌ی items. هر عضو items شامل id, type, description, quantity, unit_price, line_total, meta و created_at است.

```

1 {
2   "id": 3,
3   "number": "INV-20260129-0003",
4   "status": "issued",
5   "currency": "USD",
6   "subtotal": 10.0,
7   "tax": 0.0,
8   "total": 10.0,

```



```
9  "issued_at": "2026-01-29T08:00:00.000000Z",
10 "paid_at": null,
11 "items": [
12   {
13     "id": 11,
14     "type": "usage",
15     "description": "Chat completions usage",
16     "quantity": 1500,
17     "unit_price": 0.0005,
18     "line_total": 0.75,
19     "meta": null,
20     "created_at": "2026-01-29T08:00:00.000000Z"
21   }
22 ],
23 "pdf_url": "https://example.com/api/v1/invoices/3/pdf?...",
24 "created_at": "2026-01-29T08:00:00.000000Z",
25 "updated_at": "2026-01-29T08:00:00.000000Z"
26 }
```

کد آ-۲۶: نمونه پاسخ Invoice Detail

آ-۶-۱۱ GET /api/v1/invoices/{invoice}/pdf

کارکرد. دریافت نسخه‌ی PDF فاکتور از طریق پیوند امضا شده.

احراز هویت. به‌طور معمول از طریق امضای پیوند انجام می‌شود، نه توکن Sanctum.

ساختار درخواست. این مسیر بدنه ندارد و با signed URL فراخوانی می‌شود.

ساختار پاسخ موفق. اگر فایل آماده باشد، پاسخ دودویی PDF برمی‌گردد. اگر هنوز فایل ساخته

نشده باشد، مسیر با وضعیت 202 شیء `\{"status": "processing"}` را باز می‌گرداند و ساخت PDF را در

صف اجرا می‌کند.

آ-۷ نقاط پایانی گزارش‌ها

آ-۷-۱ GET /api/v1/reports/usage

کارکرد: گزارش مصرف با امکان گروه‌بندی و فیلتر بازه‌ی زمانی.

احراز هویت: توکن Sanctum.

ساختار درخواست: پارامترهای اختیاری to, from و group_by مقدار group_by فقط می‌تواند یکی از day, key یا provider باشد.

ساختار پاسخ موفق: آرایه‌ای از ردیف‌ها با فیلدهای group_by, group_value, metric, quantity و total_cost.

```

1 [
2   {
3     "group_by": "day",
4     "group_value": "2026-01-29",
5     "metric": "tokens_in",
6     "quantity": 100,
7     "total_cost": 0.01
8   }
9 ]

```

کد آ-۷-۲: نمونه پاسخ Usage Report

آ-۷-۲ GET /api/v1/reports/wallet-ledger

کارکرد: دفترکل تراکنش‌های کیف پول در یک بازه‌ی زمانی.

احراز هویت: توکن Sanctum.

ساختار درخواست: پارامترهای اختیاری from و to.

ساختار پاسخ موفق: آرایه‌ای از اشیای دفترکل با فیلدهای id, type, amount, reason, ref_type, meta.ref_id و created_at.

GET /api/v1/reports/invoices ۳-۷-آ

کارکرد. گزارش تجمیعی فاکتورها و وضعیت آنها.

احراز هویت. توکن Sanctum.

ساختار درخواست. پارامتر اختیاری status که یکی از مقادیر draft, issued, paid یا void را می پذیرد.

ساختار پاسخ موفق. آرایه‌ای از خلاصه‌ی فاکتورها با همان ساختار مسیر GET /api/v1/invoices.

آ-۸ نقاط پایانی مدیریت ارائه‌دهندگان**GET /api/v1/providers ۱-۸-آ**

کارکرد. دریافت فهرست ارائه‌دهندگان ثبت شده در سامانه.

احراز هویت. توکن Sanctum.

ساختار درخواست. این مسیر بدنه ندارد.

ساختار پاسخ موفق. آرایه‌ای از اشیا با فیلدهای id, name, type, base_url, status, priority, created_at و updated_at.

POST /api/v1/providers ۲-۸-آ

کارکرد. ایجاد ارائه‌دهنده‌ی جدید و ثبت پیکربندی اولیه.

احراز هویت. توکن Sanctum.

ساختار درخواست. در این مسیر، فیلدهای name و base_url اجباری هستند و فیلدهای type, priority و status می‌توانند به صورت اختیاری همراه درخواست ارسال شوند.

ساختار پاسخ موفق. یک شیء ارائه‌دهنده با همان فیلدهای مسیر GET /api/v1/providers.

```
1 POST /api/v1/providers
2 Authorization: Bearer <SANCTUM_TOKEN>
3
4 {
5   "name": "openrouter",
6   "type": "openai_compatible",
```

```
7 "base_url": "https://openrouter.ai/api/v1",
8 "api_key": "test-key",
9 "status": "active"
10 }
```

کد آ-۲۸: نمونه درخواست Create Provider

```
1 {
2   "id": 4,
3   "name": "openrouter",
4   "type": "openai_compatible",
5   "base_url": "https://openrouter.ai/api/v1",
6   "status": "active",
7   "priority": 0,
8   "created_at": "2026-01-29T08:00:00.000000Z",
9   "updated_at": "2026-01-29T08:00:00.000000Z"
10 }
```

کد آ-۲۹: نمونه پاسخ Create Provider

آ-۸-۳ GET /api/v1/providers/{provider}/models

کارکرد. دریافت مدل‌های ثبت‌شده برای یک ارائه‌دهنده.

احراز هویت. توکن Sanctum.

ساختار درخواست. شناسه‌ی عددی ارائه‌دهنده در مسیر قرار می‌گیرد.

ساختار پاسخ موفق. آرایه‌ای از اشیاء با فیلدهای id, provider_id, model_key, capabilities.

pricing_config, status, created_at و updated_at.

POST /api/v1/providers/{provider}/models ۴-۸-آ

کارکرد. ایجاد یا ثبت مدل جدید برای یک ارائه‌دهنده.

احراز هویت. توکن Sanctum.

ساختار درخواست. در این مسیر model_key اجباری است. فیلدهای capabilities.

pricing_config و status به صورت اختیاری دریافت می‌شوند.

ساختار پاسخ موفق. یک شیء مدل با همان فیلدهای مسیر فهرست مدل‌ها.

```

1 POST /api/v1/providers/4/models
2 Authorization: Bearer <SANCTUM_TOKEN>
3
4 {
5   "model_key": "gpt-4o-mini",
6   "capabilities": {
7     "chat": true
8   },
9   "pricing_config": {
10    "input_token": 0.0005,
11    "output_token": 0.001
12  }
13 }
```

کد آ-۳۰: نمونه درخواست Create Provider Model

```

1 {
2   "id": 9,
3   "provider_id": 4,
4   "model_key": "gpt-4o-mini",
5   "capabilities": {
6     "chat": true
7   },
8   "pricing_config": {
```

```

9     "input_token": 0.0005,
10    "output_token": 0.001
11  },
12  "status": "active",
13  "created_at": "2026-01-29T08:00:00.000000Z",
14  "updated_at": "2026-01-29T08:00:00.000000Z"
15 }

```

کد آ-۳۱: نمونه پاسخ Create Provider Model

آ-۹ نقاط پایانی ممیزی

آ-۹-۱ GET /api/v1/audit-logs

کارکرد: دریافت لاگ عملیات حساس برای کاربران دارای نقش مدیر.

احراز هویت: توکن Sanctum به همراه عبور از میان افزار role:admin.

ساختار درخواست: پارامترهای اختیاری actor_user_id, action, target_type, target_id و

limit.

ساختار پاسخ موفق: آرایه‌ای از اشیای ممیزی با فیلدهای id, actor_user_id, action,

target_type, target_id, meta و created_at.

واژه‌نامه‌ی فارسی به انگلیسی

Authentication	احراز هویت
Subscription	اشتراک
Provider	ارائه‌دهنده
Adapter	آداپتور
Pipeline Pattern	الگوی خط لوله
Capacity Planning	برنامه‌ریزی ظرفیت
Token	توکن
Embedding	جاسازی متن
Workflow	جریان کار
Application Programming Interface (API)	رابط برنامه‌نویسی کاربردی
One-Time Password (OTP)	رمز یک‌بار مصرف
Gateway	سامانه‌ی واسط
Client	سرویس‌گیرنده
Invoice	صورتحساب

Wallet	کیف پول
Cache	کش
Rate Limit	محدودیت نرخ
Large Language Model (LLM)	مدل زبانی بزرگ
Scalability	مقیاس‌پذیری
Playground	میدان بازی
Endpoint	نقطه‌ی پایانی
Artificial Intelligence (AI)	هوش مصنوعی

واژه‌نامه‌ی انگلیسی به فارسی

Adapter	آداپتور
API Key	کلید رابط برنامه‌نویسی
Artificial Intelligence (AI)	هوش مصنوعی
Audit Log	گزارش ممیزی
Authentication	احراز هویت
Cache	کش
Cache-Aside	کش کنار
Circuit Breaker	مدار قطعه
Client	سرویس گیرنده
Embedding	جاسازی متن
Endpoint	نقطه‌ی پایانی
Gateway	سامانه‌ی واسط
Invoice	صورتحساب / فاکتور
Large Language Model (LLM)	مدل زبانی بزرگ

Middleware	میان‌افزار
One-Time Password (OTP)	رمز یک‌بار مصرف
Pipeline Pattern	الگوی خط لوله
Playground	میدان بازی
Provider	ارائه‌دهنده
Queue	صف
Rate Limiting	محدودسازی نرخ
Repository Pattern	الگوی مخزن
Scalability	مقیاس‌پذیری
Service Layer	لایه‌ی سرویس
Subscription	اشتراک
Token	توکن
Tokenizer	توکن‌ساز
Wallet	کیف پول